Autonomous robot for dishwasher loading and unloading

by

Samson Wu

Supervisor: Andrew Goldenberg

April 7th, 2026

B.A.Sc. Thesis

Division of Engineering Science
UNIVERSITY OF TORONTO

1 Abstract

This work presents a data-efficient, perception-driven framework for autonomous dishwasher loading and unloading using a mobile manipulator in simulation. The proposed system targets the structured yet challenging task of placing diverse tableware (i.e. cups, mugs, teacups, and plates) into a dishwasher rack under realistic kitchen conditions. We investigate the effectiveness of end-to-end visual pose regression compared to a modular pipeline that integrates learned perception with geometric reasoning. Experimental results show that directly predicting object placement poses from image-based models is unreliable in low-data regimes due to the need to jointly learn viewpoint invariance, occupancy reasoning, spatial transformations, and placement preferences.

To address these challenges, we introduce a structured approach that first estimates rack occupancy using a PointNet++-based segmentation model, and subsequently selects feasible placements using prior knowledge of rack geometry encoded through RANSAC-based fitting and heuristic pruning. From successful experiments, we demonstrate the effectiveness of decomposing the task into intermediate representations. Additionally, we create a synthetic RGB-D dataset of partially loaded dishwasher configurations.

The results illustrate that modular, perception-first pipelines are strong in following rack structure and are robust to viewing angle, which end-to-end learning approaches struggle with under limited data. However, the system remains sensitive to depth quality, exhibits reduced performance under out-of-distribution configurations, and relies on assumptions about rack geometry that may limit generalization. These findings highlight the importance of integrating learned models with structural priors.

2 Acknowledgements

I would like to express my sincere gratitude to my supervisor for their guidance throughout this work. I also want to thank my parents for their unwavering support, my friends for their advice and the UofT community.

Contents

1 Abstract **1**

2 Acknowledgements **2**

3 Introduction **6**

3.1 Object Manipulation 9

4 Related Work **9**

4.1 Perception 9

4.1.1 Convolutional Neural Networks (CNNs) 9

4.1.2 Object Detection and Segmentation 10

4.2 Object Picking 11

4.2.1 Motion Planning 11

4.2.2 Grasp estimation and planning 13

4.2.3 Dishwasher Picking Object Selection (Visual Reasoning) 14

4.3 Object Placing 16

4.3.1 In hand object pose detection 16

4.3.2 Dishwasher Loading 16

4.4 Handle Manipulation 18

4.5 Global Navigation 18

4.6 Combined control, perception, and motion 20

5 Methods **20**

5.1 Robot Simulation 21

5.2 Robot Hardware 21

5.3 Indoor Scene and Dishwasher Simulation 22

5.4 Dataset creation 22

5.5 Robot Motion Planning 23

5.6 Dishwasher Picking 23

5.7 Dishwasher Loading Methods 24

5.8 Dishwasher Loading by Direct Pose Regression 25

5.9 Dishwasher Loading by Rack Modelling and Point Cloud Model 28

5.9.1 Rack Modelling 28

5.9.2 Live Occupancy Prediction with Point Cloud Model 30

5.9.3 Final Placement Prediction 31

6 Experiments and Results **32**
6.1 Rack Opening . . . 32
6.2 Rack Localization and Pose . . . 34
6.3 Dishwasher Picking . . . 35
6.4 Dishwasher Loading with Pose Regression . . . 35
6.5 Dishwasher Loading with Rack Modelling and PointNet . . . 36
6.5.1 Calculating Final Pose from Occupancy Map . . . 36
6.5.2 Autonomous Dishwasher Loading Performance . . . 37

7 Discussion **38**

8 Conclusion **41**

A Dishwasher configuration and Dataset Creation **49**
A.1 Predetermined locations . . . 49
A.2 Dataset . . . 49

B Dishwasher Loading **49**
B.1 Direct Pose Regression Deep Learning Models . . . 49
B.1.1 ResNet backbone predicting xyz position . . . 49
B.1.2 ConvNeXt backbone predicting xyz position . . . 50
B.1.3 PointNet backbone predicting xy position . . . 50
B.1.4 DINOv2 backbone predicting occupied positions . . . 50
B.2 PointNet++ Occupancy Model . . . 51
B.3 Filtering for best available position . . . 51
B.4 Autonomous Dishwasher Loading Examples . . . 52

List of Figures

1 Environment, Table Setting Goal and Object set . . . 8
2 Autonomous system overview . . . 8
3 CNN example . . . 10
4 Pointnet++ architecture . . . 11
5 RRT Motion Planning . . . 13
6 Contact-GraspNet grasp poses . . . 14

7 AnyPlace [1] two-tier architecture to do placement pose prediction using VLMs in the high-level placement location and a diffusion model for low-level pose prediction. 17
8 Mobile manipulator design 22
9 Dishwasher spawning setup 23
10 Dishwasher loading network architecture 26
11 Dishwasher picking architecture 26
12 Dishwasher Loading by 28
13 RANSAC plane fitting coloured point cloud 29
14 RANSAC plane fitting with geometric planes. 29
15 Rack support detection with RANSAC 30
16 Architecture for dishwasher loading with explicit rack modelling and occupancy network approach 33
17 Approximate rack localization 34
18 Grasp candidates with Contact-GraspNet on a small plate before pruning. . 35
19 Grasp candidate with Contact-GraspNet on mug before pruning. 35
20 Direct pose regression predictions 36
21 CNN pose regression Grad-CAM heatmap 37
22 Point cloud model sample prediction results. Each object type is represented with a different colour (Orange for mugs, Purple for teacups, Green for plates, and Blue for supports. 38
23 Final placement pose calculation 39
24 Successful dishwasher loading of two teacups 40
25 Example dataset images for middle rack 50
26 Example dataset images for bottom rack 51
27 ResNet training dynamics 52
28 ResNet pose predictions 53
29 ConvNeXt training dynamics 54
30 Sample ConvNeXt predictions 55
31 PointNet backbone for xy prediction results 56
32 DINOv2 training dynamics 56
33 Sample DINOv2 occupancy predictions 57
34 PointNet++ occupancy model training dynamics 57
35 PointNet++ occupancy model poor predictions 58
36 More final position selection examples 59
37 Autonomous dishwasher loading plate sample 59

38 Autonomous dishwasher loading failure cases 60

List of Tables

1 Pointnet++ occupancy network prediction metrics. 39
2 Autonmous loading performance by tableware type 39

3 Introduction

Recent advances in embodied AI have shown great potential for general-purpose domestic robots [2] [3] [4] [5] [6] with large Vision-Language-Action models that can do chores like cleaning, laundry. However, these are computationally and data intensive and often limited to short-horizon tasks. Notably, the task of loading and unloading a dishwasher remains largely unsolved. This is because of the challenges when dealing with a dishwasher. 1) Rack structure: The dishwasher racks have fine structure that are important to knowing how to load tableware properly into slots. This structure is not easily understood for robots, when majority of pick and place robots are trained on flat surfaces. 2) Fragile objects: The tableware are fragile, usually made of ceramic or glass thus requiring careful robot motions. 3) Pose variety: Unlike the normal flat table scenario where objects are flat on the table, the orientation of the objects in the rack are more varied. 4) Occlusion: depending on view angle, tableware may occlude other tableware.

Recent work in robotics have had some success in dishwasher loading and unloading [7] [8] [2]. A relevant prior system by J. Wong et. al. [9], only achieved task success rates of 45% for dishwasher loading and 56% for dishwasher unloading trained on human demonstrations with an imitation learning approach. Another system is able to load the dishwasher according to target configurations [10] using a learned Graph Neural Network policy. However, both previous system simply place dishware in upright poses and do not follow rack structure. The humanoid Figure robot has shown impressive demos of dishwasher loading, but success rate and generalization capabilities are unknown. Limoyo et. al. [8] learn dishwasher loading by using the reverse trajectory of picking as a loading demonstration. They learn a policy using behaviour cloning based on RGB-D images of the environment from a manual selected viewpoint. More broadly, dishwasher loading and unloading can be expanded to general tasks of perception, robot control, and motion which are the building blocks to object placing, object picking. Perception for the robot is understanding the world through sensors which is dominated in mobile manipulators by RGB-D for visual information, LiDAR, and optionally tactile sensors and microphones. Visual RGB-D information is strong for robotics

because of the constant improvement of state-of-the-art image models for object detection, segmentation, and visual reasoning. The depth part of RGB-D is used to great success in building explicit 3D point clouds of the environment. Point clouds have been used to do object pose prediction, segmentation, and grasp estimation using deep learning architectures like PointNet++ [11]. These image and point cloud models often achieve over 90% accuracy in benchmarks. Robot control is the task of adjusting the servos or motors of the robot, usually to reach or maintain a desired position, velocity, or acceleration. In modern robots that learn some kind of visuo-motor policy, the robot takes in a sensory input and learns to move the motors to a desired state. In the other case, we only care about controlling the end-effector pose to perform a manipulation task. This abstracts away the controller, where the goal now is to produce a collision-free trajectory of the arm.

In short, the key areas for improvement in the dishwasher loading and unloading tasks are reliability, handling occlusion, and ensuring loading according to rack structure. This project aims to tackle these issues by designing and developing an autonomous robot to do dishwasher loading and unloading. We propose a system to control the logic, navigation, and object manipulation to complete these tasks autonomously. Special attention will be paid to the methods and design of object manipulation.

Key objectives:

- Reliability: maximize success rate of trials
- Adhering to rack structure
- Robustness to dishwasher object configurations and occlusion
- Data efficiency

In particular, the long-horizon tasks are defined as follows. Given an indoor environment with a kitchen and a dining table, a mobile manipulator robot is placed (See Fig. 1). The first task requires the robot to develop a table setting plan consisting of locations of the tableware. At first, we consider a simple table setting plan with a combination of plates, cups, and utensils. Then the robot navigates to the dishwasher, picks out the appropriate tableware from the dishwasher, and places them on the table. The second task is the reverse process of picking tableware off the dining table and loading them into the dishwasher. The complete possible tableware set comprises large plates, small plates, cups, utensils, and small bowls (See Fig. 1). The autonomous system completes these tasks using sensors (RGB-D, LiDAR) and outputs joint control or end-effector (EE) pose for the robot to execute in real-time. With the goal of real-life deployment in mind, zero or very-minimal privileged information about the environment is given.

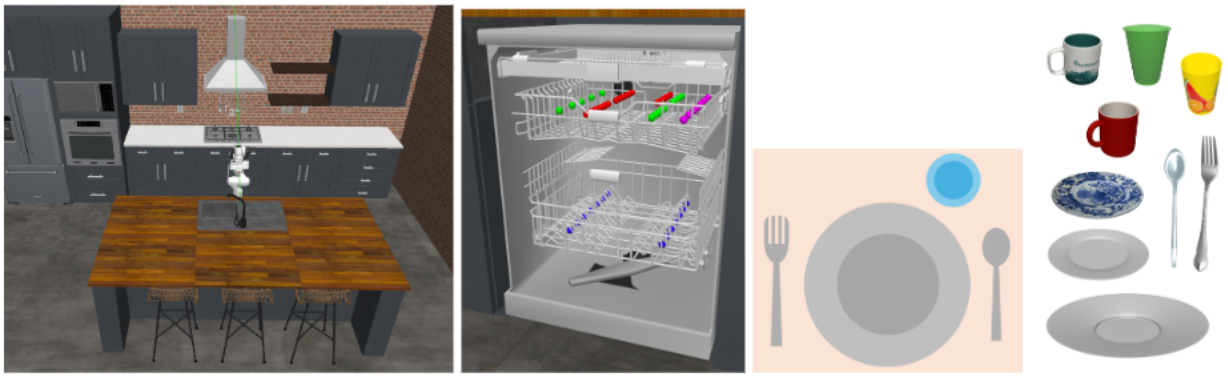


Figure 1: (Left) Example kitchen environment with dining table. (Middle left) Dishwasher model interior. (Middle right) Example goal table setting configuration. (Right) Tableware set examples.

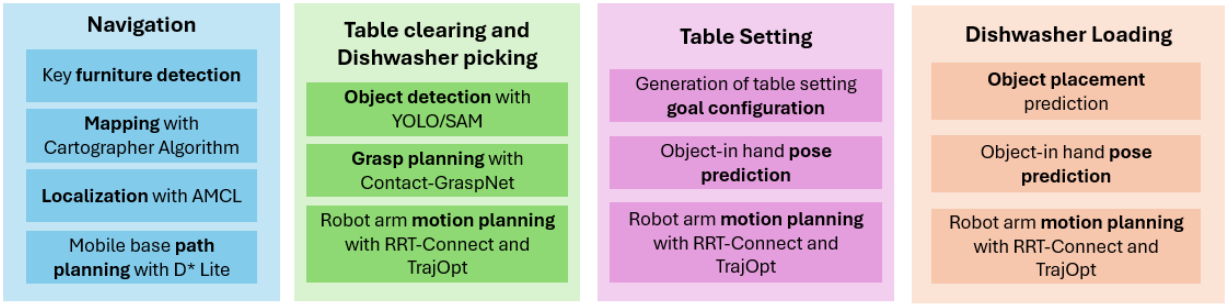


Figure 2: Autonomous system design overview.

This work presents a detailed design and implementation of the autonomous system that is successful under randomized conditions (different kitchen layouts, lighting, and furniture). Unlike many large modern systems, we solve the problem using a modular approach and classical techniques wherever reasonable. The autonomy stack can broadly be split into global navigation and tableware manipulation. The navigation stack includes perception of key furniture items (dishwasher and dining table), mapping of the room, localization of the robot in the room, and mobile base path planning. Tableware manipulation comprises table clearing, table setting, dishwasher picking, and dishwasher loading tasks. The tableware manipulation tasks all use the same robot arm motion planning system. Aside from this, table clearing and dishwasher picking tasks require object detection and selection, grasp planning. Table setting requires generating a goal configuration for the table set including tableware poses. Poses in this context consist of the specific translation and rotation from a global reference frame. Table setting and dishwasher loading both require knowledge of the object-in-hand pose to accurately place tableware. The overview of key components are illustrated in Fig. 2.

3.1 Object Manipulation

The robot is required to perform object manipulation in several key scenarios. First, it must open the dishwasher and pull out the racks, which can be formulated as a handle-manipulation task. Second, it must pick and place a variety of dishware, including plates, cups, and mugs. While handle manipulation has been extensively studied and is relatively well-understood, the problem of loading and unloading tableware presents additional challenges. Prior work has demonstrated robotic manipulation of plates and cups from generic racks; however, there is limited work addressing mobile manipulation from a conventional household dishwasher rack. This setting introduces additional complexity due to the intricate rack geometry, confined spaces, dynamic view angle, and close proximity of objects, which this thesis aims to address.

The following section explains the different components of the system and outlines the methods in the literature.

4 Related Work

4.1 Perception

4.1.1 Convolutional Neural Networks (CNNs)

Since the success of CNNs in AlexNet, they have proven to be powerful for image understanding. CNNs are composed of convolutional layers, pooling layers, and fully-connected layers (See Fig. 3. The convolutional layer transforms the input using a learned sliding kernel or filter, typically sized between 3x3 to 9x9 pixels. In general, the input to a convolutional layer is an image or a feature map. The kernel slides over the feature map and an element-wise product is computed and summed over the entire map. Across all the convolutional layers in the network, the early filters typically learn to extract low-level features like edges and corners, while the later filters typically learn high-level features like shape, or semantic meaning. A nonlinear activation is performed after the convolution layer to provide expressiveness. Pooling layers such as max-pooling is performed on the post-activation feature maps to continually reduce the dimensionality of the representation. This enables more parameters devoted to learning more high-level features of the image. Using this basic structure, CNNs are able to perform object detection, object segmentation, and pose detection, among others [12]. The following three models build off of the CNN to perform recognition tasks.

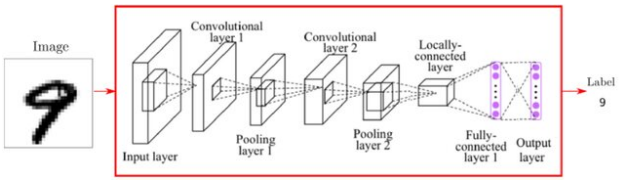


Figure 3: Simple CNN architecture example to predict handwritten digits.

4.1.2 Object Detection and Segmentation

You Only Look Once (YOLO) [13]: YOLO is a single-stage object detection neural network that predicts object locations and classes in one forward pass of a CNN and non-maximum suppression (NMS) to filter out overlapping detections. It performs well in real-time scenarios from only RGB images and is used in autonomous robots to detect furniture and objects. After fine-tuned, YOLO has been used to find handles in images, as seen in [14] [15]. YOLO is very-fast and low-resource, thus being suitable for mobile robotics. Many improvements and architectural changes have been made since the first YOLO release. Notably, YOLOv8 [16](2023) saw performance gains through a decoupled detection head an optimized backbone, and anchor-free predictions. However, these models are trained on general datasets such as COCO (Common Objects in Context) with 80 object categories. This is not sufficient for domestic kitchen use and tableware recognition. Instead, we use YOLO-World [17](2024), which is capable of general object detection based on simple text prompts. It utilizes a YOLOv8 backbone, a text encoder, and a Vision-Language Path Aggregation Network among other techniques to achieve near real-time object detection on standard hardware. For the domestic kitchen environment, YOLO-World is used for navigation and localization of key furniture (dishwasher, tables, chairs, etc.)

Segment Anything Model 3 (SAM 3) [18]: SAM 3 is a zero-shot state-of-the-art instance segmentation model developed by Facebook. It is designed to produce high-quality object masks with minimal task-specific training. Unlike traditional segmentation models that are trained for a fixed set of classes, SAM is class-agnostic and can segment arbitrary objects in an image. This is useful to find key furniture and tableware items (for example used by [1] and [19]). For this system, SAM 3 is powerful for precisely segmenting the dishwasher racks and tableware for dishwasher picking at the cost of compute time.

PointNet++ [20]: In robotics, pure RGB perception is important, however to interact accurately with the environment, knowledge of depth is crucial. Depth allows us to build an explicit 3D representation of the world, aiding in collision detection and interaction. The pioneers of using depth information is Pointnet and its successor Pointnet++ (Fig. 4. PointNet is a type of neural network that directly processes and learns on point-clouds.

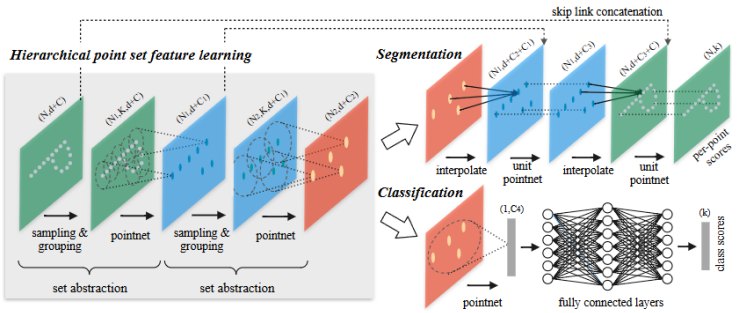


Figure 4: Pointnet++ architecture

PointNet++ improves on PointNet by hierarchically learning local geometric features from point clouds, enabling more accurate representation of complex 3D structures. PointNet's have found success doing object pose prediction, object segmentation, and grasp prediction.

Formally, a point cloud is treated as an unordered set in $\mathbb{R}^3$. They are design to work on any number of points, with higher point density leading to stronger predictions. This is due to max pooling operations. Moreover, they try to achieve translation and rotation invariance through a T-net (Transformation Network) that predicts affine transformations of points into a canonical space.

4.2 Object Picking

Object picking can be broadly divided into four subtasks, that is object localization, object pose estimation, grasp estimation, and motion planning. Task specific robots usually separate these subtasks into subsections with different algorithms to solve each one. More recently, effort has been made to combine these subtasks and do object picking in one or two-shots. These systems are usually trained on a large amount of expert human demonstrations. Notable examples are VIOLA [21], Diffusion Policy [22], OpenVLA [23], Google RT-1 [3], pi-0 [6]. However, dishwasher loading, unloading, and table setting and clearing are usually a small part of the training data. Older end-to-end policy learning includes [24], but these approaches are limited in generalization capability to novel environments. A traditional approach which uses a specialized module for each of the four subtasks can provide more flexibility in niche environments and demands fewer resources in data collection and training. We first discuss the motion planning subtask.

4.2.1 Motion Planning

Motion planning is the task of controlling a robot arm from a start pose to a target pose (usually for the EE or object-in-hand). A wide range of motion planning techniques are

available: classical sampling-based planners, visual servoing, and data-driven (deep learning) techniques.

Defining the motion planning problem: The goal of motion planning is to use an algorithm to move a mechanical system (robot) from a start state to a goal region. The result is a continuous path p in the robots state space X (usually configuration space), which also satisfies constraints not limited to obstacles or self-collision. An element of the state space is denoted as x. The task can be represented as the tuple (X_{free}, x_I, X_g), where X_{free} is the constraint-free state space, x_I is the start state, and X_g is the goal region. The state space is assumed to be a manifold that locally resembles a euclidean space $\mathbb{R}^N$ [25]. The two main constraints considered for the articulated manipulator are collision constraints and joint limit constraints.

Sampling-based planners: Sampling-based motion planners work by sampling states in the state space which are connected together (with a local planner) to find a feasible path. The most straightforward solution is Rapidly exploring random tree (RRT) + smoothing with optimization. The basic idea of RRT is that it iteratively and samples possible configurations $x \in X$, effectively creating a branching tree to explore the state space. Another option is the Probabilistic Roadmap (PRM) introduced in [26], but is not optimal in this case because it functions as a multi-query planner (the table and dishwasher racks are dynamic). Other single-query options are: Expansive Space Trees (EST), and its successor STRIDE. EST is not as sensitive to a good distance measure as RRT, but is not as simple to implement, and STRIDE is designed to perform well in high-dimensional spaces > 10, which is not needed in our case of separate 7 DoF robotic arm and mobile base. Furthermore, in manipulation experiments [27], RRT-Connect [28] outperforms other planning techniques. Many variants of RRT have been proposed; however, RRT-Connect balances speed, and practicality. RRT-Connect runs two concurrent RRTs, one from the start and one from the goal state alongside a greedy heuristic to connect the trees. RRT-Star is another powerful alternative that finds a near optimal path through iterative rewiring of nodes.

Refining Trajectory: The motion plan can be smoothed through three popular methods: 1) Shortcutting, 2) Spline fitting, and 3) Optimization-based smoothing. Shortcutting is a simple post-processing step that tries to connect two non-consecutive nodes in the path. Spline flitting simply tries to fit a spline (e.g. cubic B-splines), to nodes in the path. Optimization-based smoothing methods try to produce a path that optimizes for parameters including curvature and collisions. Common in robot manipulation is to use RRT-Connect with TrajOpt [30], a powerful optimization algorithm that excels in high-dimensions.

More recently, machine learning techniques have been found to be powerful for solving the motion planning problem as a way to inform the search direction. A direct extension

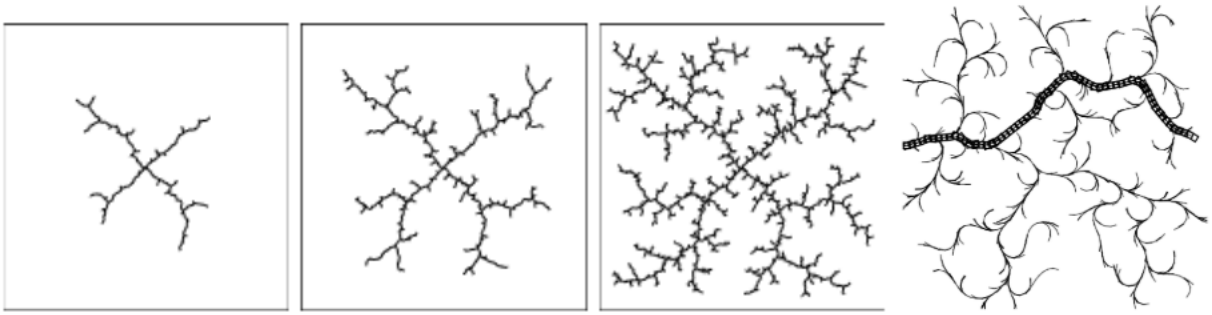


Figure 5: RRT exploring a 2D square space. (Right) A 2D projection of a 5D RRT for a kinodynamic car. [29]

of RRT is to have a biased sampler using a Conditional Variational AutoEncoder (CVAE) [31]. Due to the similarity in starting positions for the mobile base during table tasks and dishwasher tasks, learning from past experiences to speed up new motion plans is effective. However, for this system, a more complex algorithm is not necessary.

A more complex solution would be a Diffusion Motion Planner (DMP), but needs to be trained on optimal motion paths. DMP [32] learns motion planning trajectories using diffusion models as priors and proves to be successful in tests with a 7-DoF robot arm. DMP is faster on average compared to RRT-Connect + Optimization (They use CHOMP), and lower mean and variance in path length [33]. However, this method requires training time on optimal trajectories.

Similarly, cuRobo is a fast and efficient motion planner developed by NVIDIA, which have been historically used by existing autonomous robotic systems such as AnyPlace [1].

4.2.2 Grasp estimation and planning

Planning initial grasps of rigid objects is a well explored field and has found success for large variations in object types using learning-based methods. Early grasp planning using CNN's have existed since 2016, including Guo et al.'s [34] work and Pas et al.'s [35] method with Histogram-of-Gradients (HOG) and Support Vector Machine (SVM) for learning grasps from depth point-clouds. More recently, Sundermeyer et. al. introduced Contact-GraspNet [36]. They tackle the 6-DoF grasping of unknown objects in a cluttered scene directly from a depth map (partial point cloud). They generate collision free grasps and achieve over 90% success rate in cluttered scenes. Interestingly, the grasp planning starts with an observed contact point of the object and tries to predict the other side of the grasp. This reduces it to an easier problem from 6-DoF to 4-DoF, and ensures grasp is bound to actual geometry. Grasp prediction is done with an asymmetric U-shaped network based on PointNet++ [20].

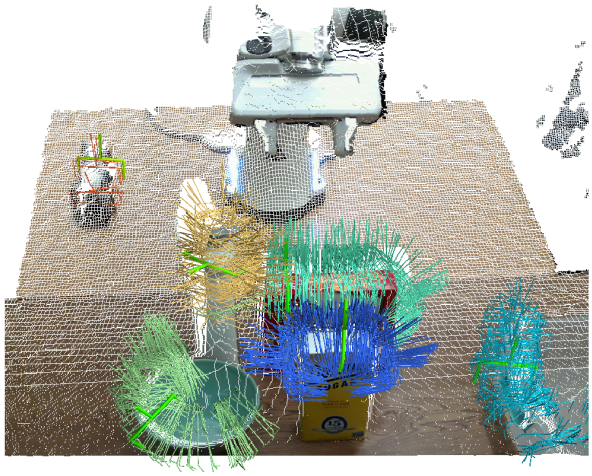


Figure 6: Contact-GraspNet example generated grasps.

Combined with a separate image segmentation network, the whole pipeline takes 0.19s for one object in a cluttered scene. To reduce the amount of false positive grasps, an accurate image segmentation such as SAM is typically used for complex environments.

Explicit 3D reconstruction approaches [37] that try to create a 3D model of the object are possible but are not necessary and computationally more expensive and slow. They often require prior knowledge of the 3D models of the objects. **Discriminative methods** train a classifier to evaluate the quality of existing grasps. Grasps candidates are generated through various sampling strategies. Dex-Net [38]] is a well known example but is designed for 2-DoF planar grasps, while [39] creates a 6-DoF grasp planner using geometric heuristics. **Generative methods:** Murali et al. [40] provides a generative method for grasp sampling using a CVAE on a (instance) segmented point cloud of the target object. They train on 126 objects and achieve 80.3% success rate in cluttered and partially occluded grasping experiments. **Other methods:** It has shown to be possible to simultaneously predict an object and a grasping pose (from a top-down view) with a shared convolutional neural network by [34]. However, dataset generation for this task is time consuming. Methods without manually defining best grasp poses is preferred. Overall, Contact-graspNet is the most promising with high accuracy and speed.

4.2.3 Dishwasher Picking Object Selection (Visual Reasoning)

Picking a dishware item is a repeated task during the long-horizon table setting task. Using a good strategy for picking affects the speed, ease, and reliability of picking. For humans, we often may not need to pull out the rack all the way, and hence take the dishes that are closest to us and easy to pick out. By contrast, the robot should pull out the rack fully to

maximize options and visibility of all the dishes in the rack. Given the rack is fully out, the robot should again prioritize ease and safety first (and speed second) to avoid collisions and in turn loud noise and possible damages. To meet safety criteria, the selection algorithm should favour items with ample clearance from surrounding objects, reducing the risk of incidental contact. For efficiency and speed, the robot should then prefer items closest to its current position, which not only shortens travel distances but also provides greater maneuverability without requiring base relocation, thus optimizing overall task time.

Towards this goal, several methods are presented and evaluated. Literature for choosing an object from an image usually involves 1) instance segmentation model trained on many labelled images, or 2) A large Vision-Language-Model (VLM) to reason about the image and choose an object.

With the instance segmentation approach, all tableware will be segmented by type. Image segmentation can be achieve using a YOLO network or SAM3. A separate tableware selection CNN small can receive the instance segmentations and output the best tableware to pick. Alternatively, a CNN model can directly predict a pixel-wise affordance map on best picking locations, conditions on the tableware type. One example of a similar system is seen in [41]

One existing VLM approach leverages Molmo / Segment Anything Model (SAM), which excels in promptable segmentation tasks using natural language queries. For instance, in robotic grasping applications, Molmo has been employed to isolate specific objects in cluttered scenes by prompting it to segment targets based on ease of access or other criteria. For each target dishware to be picked, a tailored prompt such as "Segment the plate in the dishwasher rack that is easiest to pick up and the closest" could guide Molmo to identify the most suitable item, combining semantic understanding with spatial reasoning. The VLM approach can be powerful, but out-of-the-box performance may be poor on the intricate structure of dishwasher racks.

In summary, the object picking process in this system is executed in three streamlined steps: 1) The target dishware is selected using a combination of detection models like YOLO for initial bounding or SAM for refined segmentation based on safety and proximity criteria. 2) The RGB-D image of the rack is cropped to focus solely on the chosen target, which is then fed into Contact-GraspNet to generate the most confident grasp pose, incorporating depth information for robust 6D predictions. 3) With the initial end-effector (EE) pose and the target grasp pose defined, motion planning is performed using RRT-Connect for global path finding, followed by trajectory optimization with TrajOpt to ensure smooth, collision-free execution.

4.3 Object Placing

Placing can be generally divided into two subtasks: 1) Finding the goal object placement pose, 2) Trajectory planning of end-effector and object to goal pose. This project focuses on dishwasher loading.

Concretely, we define the object placement pose as the coordinate frame of the object in a desired goal state with respect to the robot base. The canonical coordinate frame of an object typically defined at the centre of mass with an orientation arbitrarily assigned, but consistent with other objects. During object placement, the goal is to find the final pose of the object. Since we only control the robot's EE pose, the transformation between the grasped object pose and the desired end-effector pose is required. In short, the in hand object pose is needed.

4.3.1 In hand object pose detection

Pose detection is a well explored field, with numerous learning-based methods for the unknown geometry case, and CAD model alignment with ICP (Iterative Closest Point). For the scope of this project, the object pose in hand will be assumed to be known.

4.3.2 Dishwasher Loading

In dishwasher loading, finding the goal locations and poses of the tableware is more involved due to the non-trivial possible choices. Moreover, computer vision models are often trained for object recognition, rather than the absence of objects for placement. In the extreme case of messy tableware, the existing tableware may need to be reorganized before placing the new one (not considered). Following the scope of this project, the aim of the system in this step is to find a "good" spot in the dishwasher rack to place the item, alongside the pose. A "good" spot is any empty spot that may retain organization heuristics such as grouping same object types together.

One strategy is to learn how to place plates into a dishwasher rack by using the reverse of picking plates as expert demonstrations [8]. Another strategy is to use a learning-based end-to-end system to abstract-away the problem. Again, examples include the visuomotor policies of OpenVLA, VIOLA, etc.

Explicit approaches have found success in object placement tasks about hanging a mug on a rack, shelf placement, and general insertion. AnyPlace [1] is an involved method that splits the placement task into high-level and low-level (fine-grained) prediction. They are able to insert vials into racks first by predicting keypoints on possible insertion locations over a 2D image using Molmo VLM (an alternative is RoboPoint [42]). Second, they explicitly

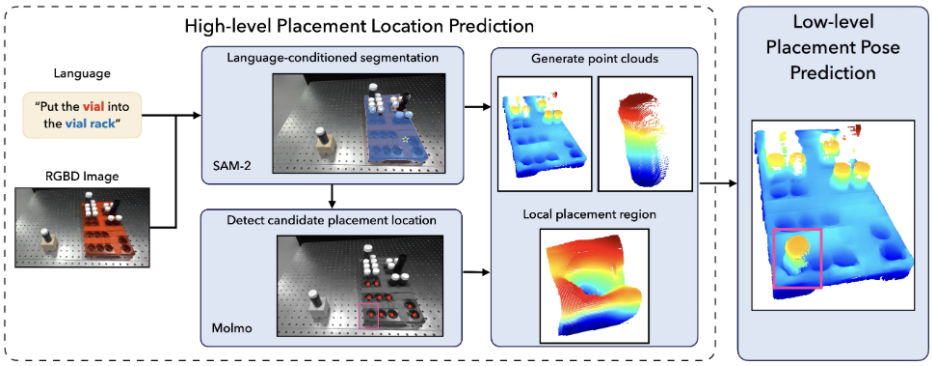


Figure 7: AnyPlace [1] two-tier architecture to do placement pose prediction using VLMs in the high-level placement location and a diffusion model for low-level pose prediction.

use the object point-cloud and predict the relative transformation from object-in-hand to the scene point-cloud (See Figure 7). This prediction is done using a Diffusion model trained on a large synthetic dataset. Overall, using a VLM to predict keypoints is applicable for dishwasher loading, however the keypoint prediction will have trouble with object occlusion (such as plates covering slots) and may be view-angle dependent. Moreover, the diffusion model training is computationally expensive. An adjacent method is Pick2Place [19] that explicitly renders the scene from different possible insertion directions using Neural Radiance Fields (NeRF) and predict a learned pixel-wise affordance map. However, this method is slow due to the requirement to capture the scene from multiple angles and generate a NeRF. Similar to the low-level diffusion model in AnyPlace [1], Pan et al. [43] has found success to learn object pair pose relations (with point-clouds) using a transformer-based approach. An implicit approach to learning object pose relations is demonstrated in Neural Descriptor Fields (NDFs) [44]. Notably, using a self-supervised approach, they can do manipulation (e.g. hanging mug on rack) on just 5-10 demonstrations. Another extension to the AnyPlace's low-level diffusion model is [45] which directly computes the final pose of an object using a diffusion model. They formulate the task as an iterative de-noising problem in 3D, trained to correct randomly perturbed poses. They are successful in shelving, hanging, and stacking tasks, while extension of this method to dishwasher placement is possible given enough RGB-D views are taken to formulate a complete point cloud of the scene.

Inspired by keypoint prediction, placement affordance maps, and pose prediction, this thesis attempts the following two methods. The first uses attempts to do direct pose prediction from RGB-D.

1. Predict pose of held object with PoseCNN [46] or ICP on known CAD models. (For simplicity, we assume this is known)

2. Predict best tableware placement pose (object pose) with custom trained CNN (See details in Sec. 5.8)

3. Place according to goal pose using motion planning (See Sec. 4.2.1) a small height above the desired spot, and then move down to place.

The second method is as follows:

1. During first deployment, form a complete point cloud of dishwasher racks and detect important support structures.

   (a) Generate possible placement slots and positions based on standard dishware dimensions.

2. During real-time deployment, predict pose of held object

3. Predict occupied positions in the rack using a Pointnet type architecture.

4. Predict best available position in rack based on occupied positions and generated possible placement positions.

5. Place according to goal pose (same as method 1).

4.4 Handle Manipulation

Hsu et al. propose KINSCENE [14], has proposed a method to explicitly learn the articulation model for doors through interaction with the environment. They try to predict the vector for rotation and prismatic joints. Joints are then manipulated by grasping the detected handle (dedicated network for handle detection) and following the calculated trajectory. They achieve 73% success rate in scene manipulation. For handle detection, they achieve high success using YOLO detection trained on the DoorDetect dataset. A similar method with a focus on autonomous door operation is illustrated in [15].

4.5 Global Navigation

In the context of global navigation, the mobile manipulator robot needs to move in a domestic environment from the dishwasher and dining table (or other kind of table). Using vision-based sensors (colour and depth), the robot will understand the environment (mapping), avoid obstacles, localize, and navigate. Localization is the task of learning where one is in the environment. Mapping is the task of representing the environment, often to aid

navigation. Navigation is not a large focus of this thesis, so the following section will give a brief overview of typical global navigation techniques seen in mobile manipulators.

There are several requirements for global navigation:

- Traverse between dishwasher and dining table (Learn locations of dishwasher and table)
- Avoid collisions
- Respect kinematic constraints of mobile base
- Ensure reachability in target state for dishwasher and table tasks

Related work in this area highlights various approaches to indoor navigation for similar robotic systems. For instance, [47] has demonstrated effective indoor navigation by combining 2D LiDAR and RGB-D cameras, employing Cartographer for Simultaneous Localization and Mapping (SLAM) and Adaptive Monte Carlo Localization (AMCL) for positioning. Cartographer provides real-time SLAM capabilities in both 2D and 3D across diverse platforms and sensor setups, generating an occupancy map from processed RGB-D and 2D LiDAR data. Online localization is then handled by AMCL, a probabilistic system designed for robust positioning.

Another notable example is the TIAGo service robot, which leverages the ROS navigation stack, including GMapping and AMCL for mapping and localization [48]. Global path planning in this setup is accomplished using the A* algorithm, while local planning relies on Dynamic Window Approach or Timed Elastic Band methods.

Semantic mapping plays a crucial role in identifying key targets like the dishwasher and dining table. Traditionally, this can be achieved by first using standard 2D/3D SLAM algorithms, such as GMapping, to construct a metric map (e.g., an occupancy grid), which is then transformed into a graph-based topological map [49]. Object recognition within these maps can be facilitated by modern real-time detection networks like YOLO. Advancements in deep learning have further enabled pure RGB or RGB-D SLAM systems, such as DROID-SLAM [50]. In the KINSCENE project [14] for example, 2D and 3D SLAM techniques are used to build a static map of an indoor kitchen environment. The robot is manually navigated through the scene to produce a 3D model via DROID-SLAM, with Nautilus employed for 2D navigation mapping and autonomous loop closure. By using a semantic map, we can navigate using locations of the dishwasher and dining table as start and goal locations.

Global path planning: Given the non-holonomic vehicle being used in simulation, there are several classical and sampling-based planners to be considered. Classical global planners include A*, D*, and D*-lite. Sampling-based planners such as PRM and RRT

mentioned in Section 4.2.1 with kinodynamic constraints are strong for global path planning. Due to the search space being a 2D domestic room and 2-DoF mobile base movement, the benefits of PRM and RRT are unused. Splitting up the room into a grid and performing a classical global planning technique is more than sufficient. In the classical global planners, A* is not designed to hand dynamic environments, and D* Lite is an improvement over D* in efficiency and simplicity. Therefore, D* Lite will be the algorithm used. To ensure reachability of tableware on the dining table, locations can be chosen using the 2D map of the current table state and distance metrics.

4.6 Combined control, perception, and motion

Hybrid Robot Actions for Imitation Learning (HYDRA) [51] and Diffusion Policy [22] are techniques that use learned policies to perform one-shot perception-to-action in real-time. These techniques are out of the scope of this project due to the difficulty in collecting high quality demonstrations, but could be considered in the future. HYDRA, presents an imitation learning method that can complete dishware tasks including: picking a spoon, placing a spoon, grasping a plate, inserting a plate, picking a mug, and hanging a mug. They achieve 100% on spoon tasks, 90% on plate tasks, and 80% on picking a mug. They train their model on 100 demonstrations per task. An adjacent method in [9] also uses imitation learning to learn table setting and dishwasher tasks using their novel teleoperation framework. However, they only achieve 45-50% success in dishwasher loading and unloading. Chi et. al.'s [22] method learns a policy with diffusion to complete tasks such as flipping a mug and hanging a tool with $\tilde{1}00$ demonstrations per task. Interestingly, the diffusion policy learns multimodal behaviour (i.e. output multiple ways to manipulate an object) and generates smooth paths. VIOLA [21] presents an imitation learning method to complete kitchen tasks like table setting with 100+ human demonstrations. They learn a visuo-motor policy with a transformer-based architecture, achieving 60-76.7% accuracy on table setting tasks.

5 Methods

The following section will detail the design of the system, the creation of the dataset, and training of computer vision models.

5.1 Robot Simulation

To support the simulation of the system, the Robocasa (2023) [52] library is used on a Windows system with an RTX 3060 GPU. There exist many options for simulation, including NVIDIA ISAAC, Gazebo, and Moveit on top of ROS/ROS 2. Nvidia ISAAC is the most powerful simulation software with photo-realistic rendering using a high-fidelity GPU-based PhysX engine. It is also suitable for deploying to real-world robots. However, the compute requirements are beyond standard consumer hardware. Moveit is a strong robotics simulation framework built ontop of ROS that emcompasses all the important functionality. Robocasa is a library built on top of Robosuite which is built on the Mujoco framework. Mujoco is an open-source general purpose physics engine which can be run on Windows OS, and is relatively lightweight. Robosuite is a simulation framework utilizing Mujoco that provides all the necessary robot models, environments, and robot controllers to do simulation. Overall, Robocasa provides a vast array of kitchen environments of multiple layouts, interactable furniture, and objects. This gives a great starting point for learning and simulating dishwasher tasks.

5.2 Robot Hardware

The main requirements for the design are global navigation, a precise robot arm, a suitable gripper, a large vertical range of motion, and sufficient sensors to help with localization and perception. With these considerations and the use of the RoboCasa framework, the robot design is detailed below and in Figure 8. The mobile base is the Omron LD-60, with 2 wheels and 2 DoF (non-holonomic). The robot arm is a 7-DoF Panda arm commonly used in robotics research. A large vertical range of motion is needed to reach objects from a table surface, and the lowest dishwasher rack (around 0.2 to 1.2 metres). To this end, a linearly actuated lift column is installed on the base to meet this range of motion. An impactful design choice is that of the end-effector which needs to accurately and reliably hold tableware. A secondary requirement is the need for a small-footprint end-effector to allow for maximum maneuverability inside a complex dishwasher rack. To maximize compatibility with existing robotics libraries and research, the Panda parallel gripper is used. Importantly, Contact grasp-net does grasp generation assuming a parallel gripper, so to reduce the need for re-training the model, we use a parallel gripper. Finally, to minimize grasp force and slippage, a rubber tip is approximated in simulation.

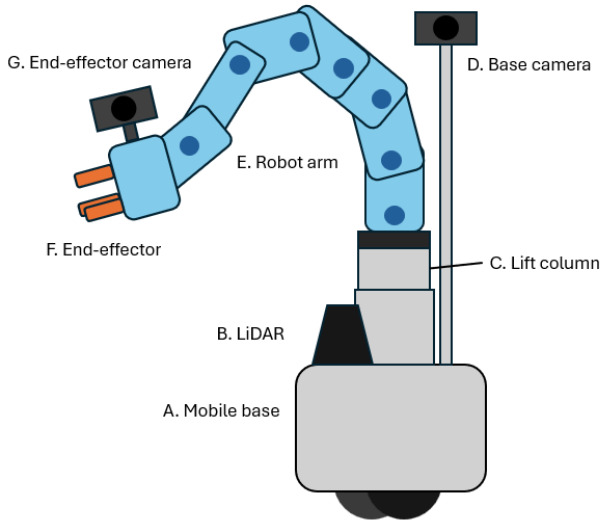


Figure 8: Mobile manipulator design overview. A. Omron LD-60 mobile base. B. 2D-LiDAR. C. Vertical lift column. D. Camera attached to base. E. 7-DoF robot arm. F. End-effector.

5.3 Indoor Scene and Dishwasher Simulation

An indoor scene with a kitchen, dishwasher, and dining table is created in simulation using pre-made kitchen 3D models and layouts provided by Robocasa. The scene is simulated using Mujoco, which uses static lighting. A detailed model of the dishwasher is taken from online, which has standard rack structure. The dishwasher has three extendable racks. The bottom rack stores large plates and bowls. The middle rack holds small plates, cups, and mugs. The top rack is short in height, containing utensils such as spoons, forks, and knives.

5.4 Dataset creation

There are two main models to be trained based on RGB-D images of the dishwasher in different configurations. To maximize the robustness and generalization of the model, images are generated from multiple angles, in random lighting conditions, and with random configurations. During dishware spawning, the type of dishware is limited to a set of (predetermined) locations which follow the rack support structure (More information of predetermined location selection in Appendix). Specifically, each spawn location is indexed to allow for simple integer ground truth labels. The locations are visualized in Figure 9. Each dishware is randomized slightly in its orientation and position (a few millimetres and degrees). Overall, 60 rack configurations are generated, with 10 random camera angles, resulting in 1800 RGB-D images at 480x480 resolution. Ground truth labels for occupied positions, empty positions, and dishware poses are stored in a JSON file. To support training of a pose pre-

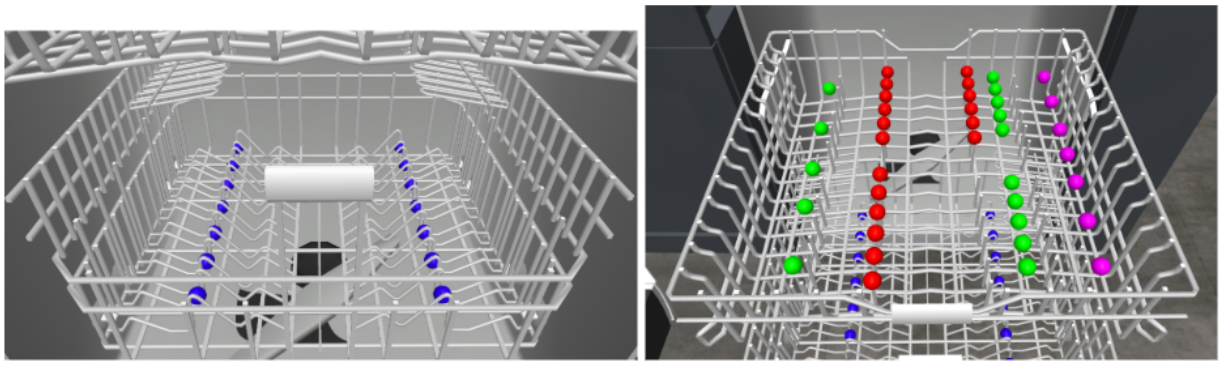


Figure 9: (Left) Bottom dishwasher rack spawning locations marked in blue for large plates. (Right) Middle dishwasher rack spawning locations marked in green, red, and magenta. Left side green markers are the location for mugs. Right side green markers are small plate spawns. Red markers indicate possible coffee cup or tea cup spawns. Magenta markers show cup spawns.

diction model, additional ground truth labels are manually created for the most preferred placement positions. (See Appendix for example data).

5.5 Robot Motion Planning

Because the pick and place strategies are focused on producing a goal end-effector pose rather than an automated control policy, robust motion planning is required to reach the goal pose, without collision of the dishwasher rack. As mentioned in Section 2, the RRT* algorithm is a simple one-time query motion planner that outputs close-to-optimal paths. In practice for simple trajectories, RRT* takes < 0.05 seconds to find a solution. The initial generated path is smoothed using cubic spline interpolation, with sampling density scaled to total distance travelled in the task space. Note that if hardware setup allows, cuRobo is a faster and better alternative because of its learned control that minimizes the path complexity (length and jerk).

5.6 Dishwasher Picking

As outlined in 4.2.3, the dishwasher picking pipeline is inspired by [8] pipeline. Assuming the robot has already opened the dishwasher door, and a target object type is provided. A detailed outline of the pipeline is written below and visualized in Fig. 11

1. Based on a standard mapping of object type to rack, the bottom, middle, or upper rack is chosen.

2. The rack is pulled out based on techniques described in 6.1.

3. The robot takes one or two RGB-D images of the rack (looking towards the centre of the rack) using the EE camera or base camera. The RGB-D are combined into one point cloud (if applicable), and the background points are cropped out if they are too close or too far.

4. The target object is segmented from one or two RGB images using SAM 3.

5. Contact grasp-net is used to predict best grasp candidates of the segmented object, given the point cloud and segmented point cloud.

6. Grasps are filtered to exclude grasps that are too close to the rack walls.

7. Grasp with highest confidence score is executed with RRT* motion planning.

5.7 Dishwasher Loading Methods

There are multiple ways to do dishwasher loading, assuming the goal is a neat organization of tableware following racks structure. **1) RL.** One such method is reinforcement learning, where the robot is controlled by a policy that tries to learn how to place tableware given an image at each time step. This comes with many challenges, with the major one being avoiding obstacle avoidance. **2) Supervised Learning.** Another method is to do a supervised learning approach with the robot control policy and learn by demonstration. Due to a focus on computer vision and limited resources, this method is not pursued. **3) Placement pose regression** Third, is to predict the final pose of the object-in-hand or similarly the end-effector, and complete the placement using a trajectory planning system. The difficulty with this method is that the motion planning and placement are treated separately, and the motion planner has to find a smart solution that may not be possible based on the robot arms current position. For example, if a mug is to be placed upside down in the rack, but most of the rack is already full, then there may not be enough empty space for the robot arm to manoeuvre. The potential difficulty of motion planning in the decoupled approach can be mitigated in two ways. The first is an end-effector with a small footprint that can fit in small spaces. The second is to force the robot to grasp the object in a pose that is close to how it should be placed in the rack. For cups and mugs, this means grasping it upside down, and for plates it is sideways with the EE pointing downwards. We use this mitigation strategy to simplify the solution for the motion planner.

Exploring method (3), there are multiple possible strategies to predict the final pose of the object in rack. One way is following the Diffusion method presented in Section 5, where given a 3D representation of the scene, a diffusion model is trained to fit an object in an

empty space. The issue with this method is that the real-time diffusion inference is slower than the alternative CNN. Another way is following a procedure similar to AnyPlace, where they split up the pose prediction problem into two substeps; 1) position prediction (3 DoF) and 2) orientation prediction with local iterative refinement. The major issue with applying this to dishwasher loading is that the position prediction they do with the VLM Molmo will not be able to reliably predict possible keypoints because of occlusion between tableware. Fine-tuning Molmo may help accuracy, but is not at all guaranteed and is out of the scope of this thesis due to compute requirements. This thesis explores two alternative methods for final pose prediction; 1) direct pose regression with an deep image neural network, and 2) explicit rack modelling and occupancy prediction by a deep point cloud neural network.

5.8 Dishwasher Loading by Direct Pose Regression

The setup for image-based direct pose regression is as follows: Given an initial rack configuration, one-or-more photos (RGB-D) are taken of the rack. (Abstracting away the end-effector) the goal is to predict a viable object pose in 3D space, that is position and orientation (minimally specified by 6 numbers), in either the rack frame or the robot base frame. Prior literature has shown the pose prediction problem can be solved with high accuracy from even just monocular RGB (PoseNet, etc.). Although there is a distinction in that prior models usually predict object pose, and not possible placement poses. To achieve this, an image-based neural network model is trained to predict the loaded pose in the rack frame. The learning-based model architecture is seen in Figure 10. The model is trained on randomized view of a partially loaded dishwasher, with ground truth labels on pose of a selected tableware in the rack frame.

The feature extraction block (in green) can be replaced by any kind of image-based model. In this paper, pre-trained ResNet-34, ConvNeXt-Medium, DINOv2, and PointNet++ are evaluated. The goal of the feature extraction block is to learn useful features of the image that pertain to object placement; i.e. existing objects, empty spaces, and rack support structure. The fully connected block (in blue) attempts to learn the pose of an object-to-be-placed as a 7-length vector, $\hat{y} \in \mathbb{R}^7$. Additional context is provided to the FC block to aid in prediction. The object type (of the currently grasped object) is passed as a one-hot vector and optionally encoded, alongside any other information. The object type is a size 8 one-hot vector to encode up to 8 object types. (Network details in the Appendix 8).

To split up the task of position and orientation prediction, multiple parallel FC blocks are also evaluated. A further splitting is also evaluated, with each FC block predicting either position or orientation.

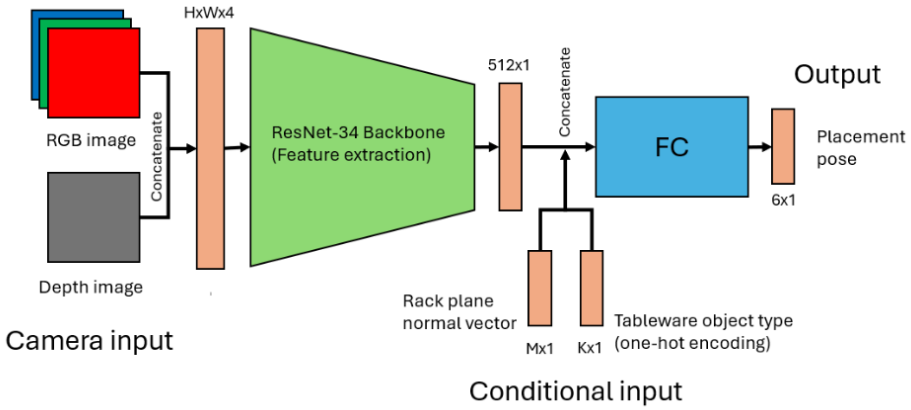


Figure 10: Neural network design for tableware placement pose prediction. The network takes in RGB-D input and object type (one-hot encoded) and outputs placement pose. ResNet is used for image feature extraction whose output is fused with known conditional input including object type and optionally rack plane geometry. FC are the fully connected layers that do regression on the best object pose.

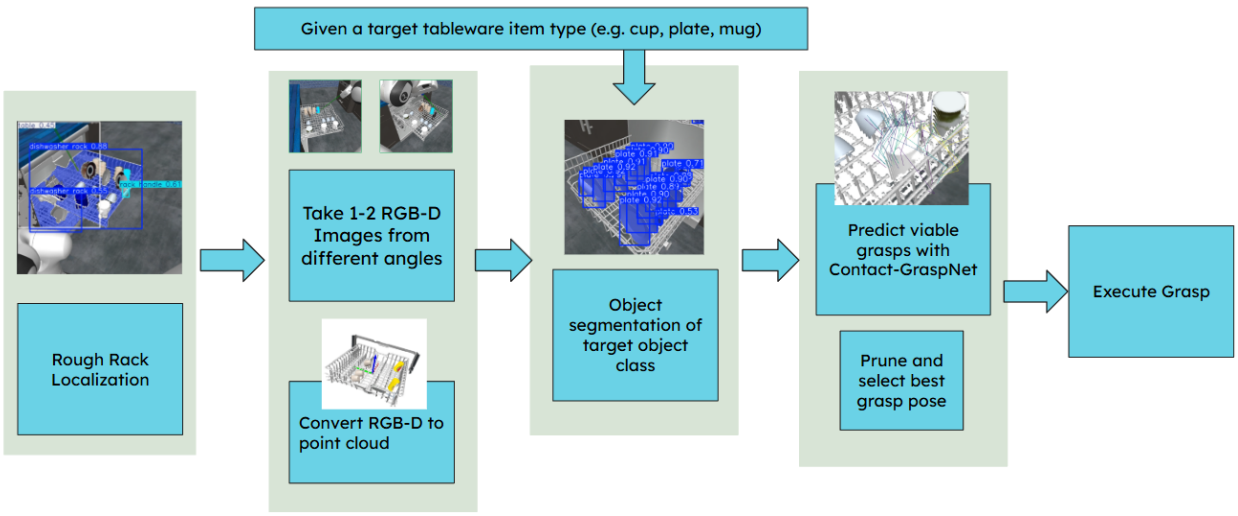


Figure 11: Dishwasher picking autonomous architecture.

Formally, we define the RGB and depth input as tensors: Let

$$\mathbf{I} \in \mathbb{R}^{H \times W \times 3}$$

be the RGB image and

$$\mathbf{D} \in \mathbb{R}^{H \times W \times 1}$$

be the associated depth image. Let

$$\mathbf{X} \in \mathbb{R}^{H \times W \times 4}$$

be the concatenation of RGB and Depth.

Let the ground truth pose be $\hat{\mathbf{y}}$ and the predicted pose be $\mathbf{y}$

$$\hat{\mathbf{y}} = \begin{bmatrix} \mathbf{p_g} \\ \mathbf{q_g} \end{bmatrix}, \quad \mathbf{y} = \begin{bmatrix} \mathbf{p_g} \\ \mathbf{q_g} \end{bmatrix}$$

where $\mathbf{p_g} \in \mathbb{R}^3$ is the goal position and $\mathbf{q_g} \in \mathbb{R}^4$ is the goal orientation as a quaternion. We use quaternion because although it is an additional parameter to predict, the 4D space is continuous (unlike euler angles) and allows the network to smoothly and stably learn with gradient descent.

For the multiple-head version, we denote each ground truth target as $\hat{\mathbf{y}}_\mathbf{i}$ where $i = 1, 2, 3$. $\hat{\mathbf{y}}_\mathbf{1}$ is the most preferable placement and $\hat{\mathbf{y}}_\mathbf{3}$ is the least preferable placement.

The entire network is trained using a few different loss functions and ground truth labels.

$$L_1 = L1 \ loss = |\hat{\mathbf{y}} - \mathbf{y}|$$

$$L_2 = L2 \ loss = (\hat{\mathbf{y}} - \mathbf{y})^2$$

$$L_3 = Huber \ Loss = \begin{cases} \frac{1}{2}(\mathbf{y} - \hat{\mathbf{y}})^2 & \text{for } |\mathbf{y} - \hat{\mathbf{y}}| \leq \delta, \\ \delta \cdot \left(|\mathbf{y} - \hat{\mathbf{y}}| - \frac{1}{2}\delta\right), & \text{otherwise.} \end{cases}$$

where δ is the threshold between quadratic and linear loss, usually set to 1.

Network 1: ResNet + one-head predicting $\hat{\mathbf{y}}_\mathbf{1}$

$$L = L_1(p_g) + L_2(q_g)$$

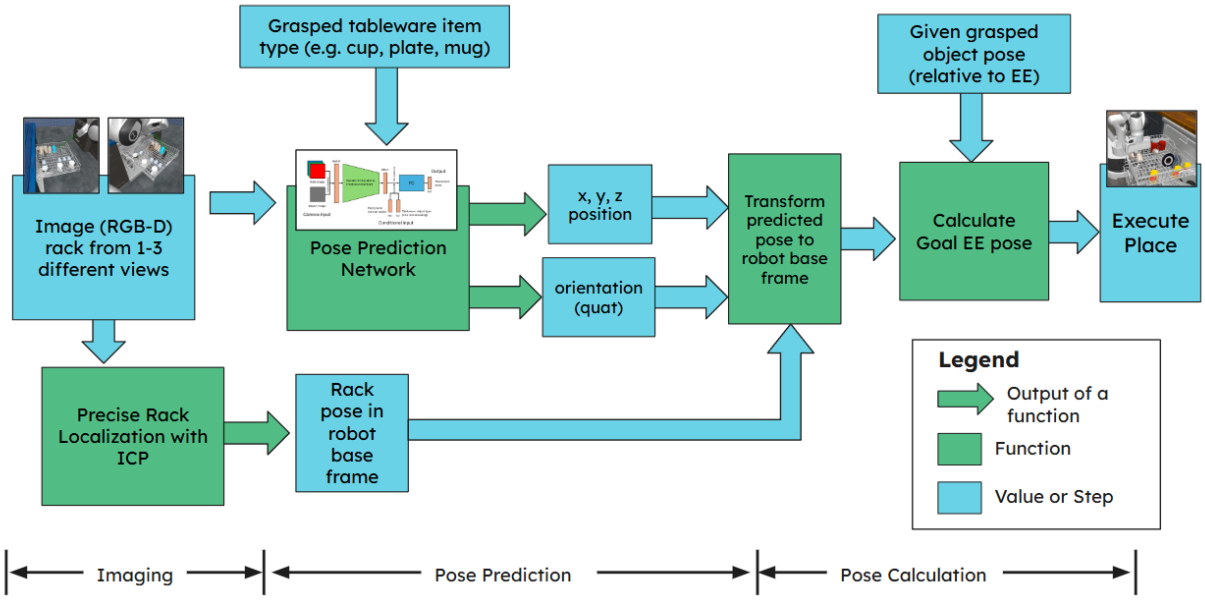


Figure 12: Dishwasher loading pipeline architecture using direct pose regression from RGB-D image(s). After the model predicting pose in the rack, there are two extra transformation steps. First is to transform it into the robot base frame. Second is to calculate a small offset pose of the EE depending on the object pose in-hand.

Network 2: ResNet + 3-head predicting $\hat{\mathbf{y}}_1$, $\hat{\mathbf{y}}_2$, and $\hat{\mathbf{y}}_3$

$$L = \sum_{i=1}^{3} (L_1(p_{g,i}) + L_2(q_{g,i}))$$

The fully autonomous pipeline using the pose regression network is depicted in Fig. 12.

5.9 Dishwasher Loading by Rack Modelling and Point Cloud Model

The idea of explicit rack modelling method was formed to combat the difficulty of direct pose regression outlined in the previous section. To load tableware following rack structure, the most straightforward way is to gain knowledge of the rack structure. Thus, we introduce an initial explicit rack modelling step during first-time deployment into a new environment (with a new dishwasher). The end goal of the racking modelling is to output possible plate slots and cup positions which precisely conform to the specific rack.

5.9.1 Rack Modelling

Firstly, we take a detailed 'scan' of the entire rack and form a detailed point cloud. Formally, 10 random views of the empty rack are taken in RGB-D and converted into point clouds, while

recording the relative camera poses. Each point cloud is cropped to remove the background points using knowledge of the position and size of the rack with SAM 3 and the depth image. The 10 point clouds are then registered using ICP (Iterative closest point) using relative camera poses as the initialization. Then, the floor and walls of the rack are retrieved using RANSAC plane fitting (with plane-thickness of 1 cm). Assuming the floor is the largest surface of the rack, the floor plane is extracted as the plane with the most points. (Seen in Fig 13 and 14)

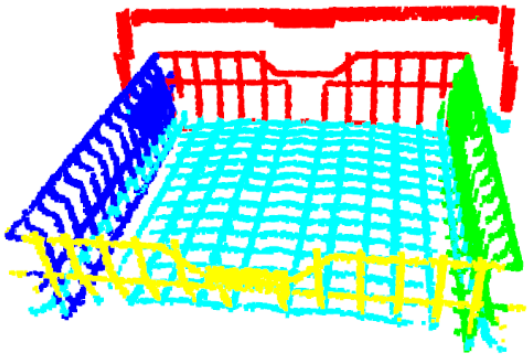


Figure 13: RANSAC plane detection shown as coloured point cloud. Each colour is a detected plane.

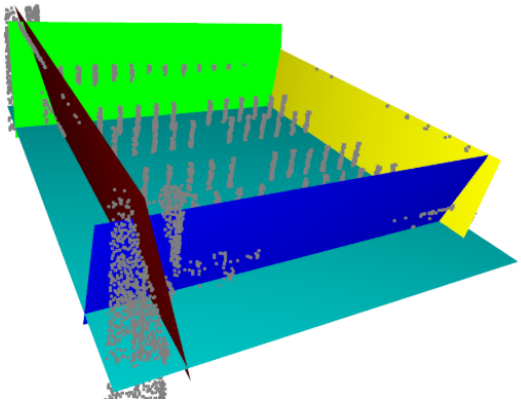


Figure 14: Simplified plane representation as a result of plane detection.

Then, the 'working area' of the rack is extracted using the intersection of the wall planes with the floor plane. We define the working area as the area of the rack without walls. To reduce the number of point clouds and increase efficiency, the walls are cropped using the predicted working area. Likewise, the floor is cropped using the fitted floor plane parameters.

Next, we try to find all the support structures in the rack. Without loss of generalization to standard racks, we assume the supports have some significant vertical component. Vertical supports are found using a RANSAC, with restrictions on support radius and length (See Fig 15). To speed up the process, the vertical support fitting is ran on patches of the rack (10 cm x 10 cm). Due to randomness of RANSAC, we run this process multiple times until a consistent result is achieved according to number of supports fitted.

Considering the two main purposes of supports being plate slots and cup hanging, we find possible placement positions. Plates slots are approximated as rectangles defined by parallel supports, and cup spot are approximated as circles with edge-thickness. Given the working area and supports, we complete a pre-processing step where the supports are projected into a 2D top-down view. **Plate slots:** Using tuned parameters, a brute force approach is taken to search for combinations of 4 supports that are parallel and aligned. **Cup spots:** Using tuned parameter, a similar brute force approach is taken to find possible circle placements

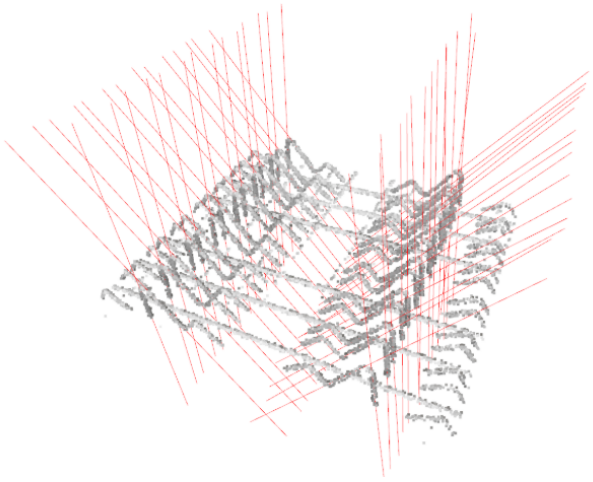


Figure 15: Rack support detection with RANSAC line fitting. This is the middle rack with cropped walls. Triangle support planes are approximated by taking the intersection of lines and the floor plane.

that contain one-or-more vertical support rods. We also do another search for possible cup placements that are not hanging on vertical support rods.

The search for plate slots and cup spots are run on different sizes of rectangles and circles, according to standard plate and cup dimensions. Finally, during this process, the 2D top-down placement positions are recorded as the centre of rectangles and centre of circles.

5.9.2 Live Occupancy Prediction with Point Cloud Model

During real-time deployment, the goal is to predict the best placement position and orientation based on the occupied and available positions in the rack. To characterize this environment, a point cloud model processes multi-view RGB-D inputs to determine occupied regions. This task is framed as a binary semantic segmentation problem on the rack's floor plane, distinguishing between occupied and available regions. To ensure geometric consistency for the segmentation backbone, the original floor points are cropped and replaced with a synthesized, uniformly dense floor. This standardized representation facilitates precise top-down labeling of occupancy masks. While the PointNet++ architecture is inherently capable of high-fidelity object recognition, this specific formulation requires the model to project 3D object geometries onto the 2D floor surface. Ultimately, optimizing for a segmentation objective offers superior convergence and stability compared to direct pose regression, while enabling the leverage of features from pre-trained PointNet++ encoders.

Formally, we setup the occupancy prediction problem as follows. Let $M_i \subseteq \mathbb{R}^3$ be the point cloud created through direct conversion from RGB-D, where i is the view number $(i = 1, ..., v,\ v \in Z,\ v \leq 3)$. M_i's are merged together to $M \subseteq \mathbb{R}^3$ using ICP for point cloud

registration.

Let $\hat{M} \subseteq \mathbb{R}^3$ be the detailed point cloud found offline from the first-time rack modelling pipeline. M and $\hat{M}$ are merged together using ICP to recover any lost rack structure from occlusion. Due to possible symmetry of the rack, four initializations of ICP are completed at 90 degree intervals, with the highest fitness score taken.

Let $M_{crop} = M - M_{walls} - M_{floor}$, be the point cloud without floor points and wall points Let $M_p = M_{crop} + M_{uniformfloor}$, be the point cloud with a uniformly dense floor.

The ground truth labels of occupied classes are labelled by setting all points in a cuboid around each object to occupied. The ground truth labels of the support structure class are labelled using the line fitting solutions from the rack modelling procedure. In total we consider 6 segmentation classes: 0: background, 1: occupied cup, 2: occupied plate, 3: occupied teacup, 4: occupied mug, 5: support

The PointNet++ network is trained to receive M_p as input, and outputs a class label for every point in the input. The PointNet++ architecture is shown in Fig 4. The PointNet++ is initialized using pre-trained weights from the part segmentation provided by the original paper (trained on ScanNet, an indoor scene segmentation dataset). It is fine-tuned using the custom generated dataset, where M_p is created from 2-random views of the rack. The network predicts the probability of each point being from a segmentation class. The loss function is weighted cross-entropy loss on the class probabilities. Initially, the classes are weighted by inverse class frequency.

$$Weighted\ CE\ Loss = -\frac{\sum_{i=1}^{N}\sum_{c=1}^{C} w_c y_{i,c} \log(\hat{y_{i,c}})}{\sum_{i=1}^{N}\sum_{c=1}^{C} w_c y_{i,c}}$$

where C is the number of classes, N is the number of points (for a point cloud), w_c is the weight for class c , $y_{i,c}$ is a binary indicator which is 1 if point i has true class c, and $\hat{y_{i,c}}$ is the predicted probability for class c.

5.9.3 Final Placement Prediction

Given the point cloud segmentation output of the PointNet++ model, we generate a 2D top-down picture of what points are occupied. This is simply created by cropping the point cloud to only contain points near the floor (-3 to +3 cm), denoted $M_{p,floor}$ and then projecting to the x-y plane by truncating the z-values. All areas of the remaining point cloud with occupied classification are considered to be occupied by an object. Let the set of points in $M_{p,floor}$ which are classified as occupied by $M_{p,occ}$. Taking the possible placement shapes—rectangles for plates, and circles for cups from the rack modelling step—we classify each

shape as occupied using a simple density measure. Formally, we define

$$D_{avg} = \text{average density of point cloud } M_{p,floor}$$

$$D_{threshold} = c \cdot D_{avg}$$

where c is some constant between 0.1 and 0.5, tuned through experimentation.

$$D_i = density\ of\ shape\ i$$

where i denotes the shape index.
A shape is considered occupied if $D_i > D_{threshold}$.

Finally, an available shape is chosen which is 1) closest to an object of the same type or 2) closest to the robot's location if no objects of the same type are in the rack. These heuristics aim to maintain organization and ease of placement. The target pose of the object is the midpoint of the shape with a height of 5-15 cm above the rack floor depending on the object type.

The complete autonomous pipeline for dishwasher loading using the PointNet network is illustrated in Fig. 16

6 Experiments and Results

We evaluate each subsystem separately, and then together in a completed pipeline. Special attention is put onto trained perception neural networks for dishwasher loading.

6.1 Rack Opening

In our experiments, we do not test dishwasher and rack opening on handles, but provide a theoretical design and evaluation instead. Assuming the robot has performed navigation in front of the dishwasher, the handle can be detected using one of three methods. 1) a fine-tuned YOLOv8 or similar on a handles dataset which requires the offline computation time and dataset collection, but provides real-time detection. 2) YOLO-World conditioned on "dishwasher handle" and "rack handle" classes. This is slightly slower than YOLOv8 but is a zero-shot solution with decent performance. 3) SAM 3 prompted with "dishwasher handle" and "rack handle" classes. Like YOLO-World, SAM 3 is a zero-shot solution and is more accurate, but comes at slower prediction time. Empirically, YOLO-World is accurate on handle prediction, and minorly worse than SAM 3. Balancing efficiency and compute,

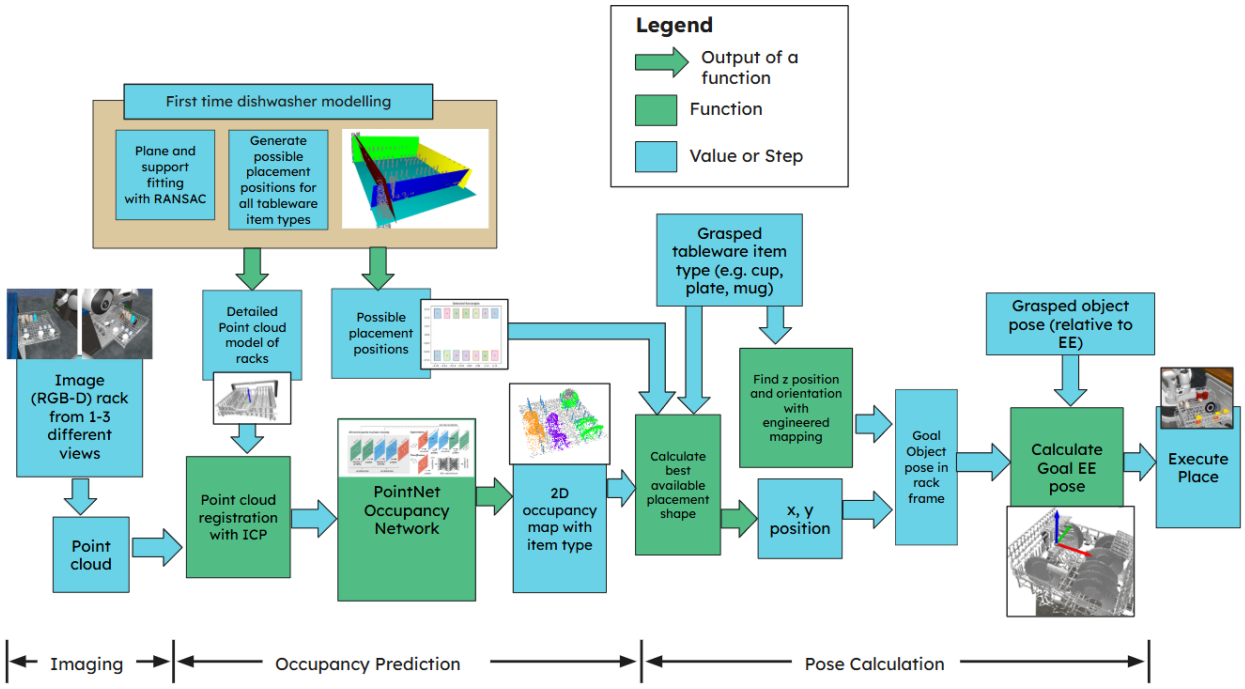


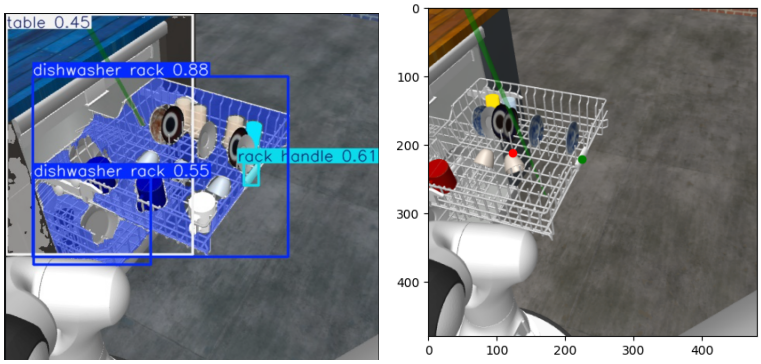
Figure 16: An overview of all the steps required to solve for the final EE pose, before commanding the robot. The first time dishwasher modelling is can be seen as an "offline" step. During real-time deployment, the autonomous system images the rack and pulls in the rack modelling details to aid in prediction.

YOLO-World is the best choice.

After handle detection is complete, the handle centre can be localized in the robot's base frame using the depth image. Dishwasher opening can be completed using a precomputed trajectory. Rack opening can similarly be completed using a precomputed trajectory that takes into account rack depth. Rack closing is the easiest operation with the least precision required. It can similarly be done with a precomputed trajectory based on rack depth.

6.2 Rack Localization and Pose

As a preparation step before dishwasher picking, a rough localization of the rack is done. High precision is not required because the grasping estimation network is independent of the rack pose. The localization of the rack is only for positioning the robot and the camera to be in a good location before picking. SAM 3 is used to do zero-shot instance segmentation of the dishwasher rack and handle. Filtering the highest confidence predictions, the average position of the rack and handle segmentation masks are calculated. A vector is formed to define the direction of the rack in the camera frame. Following the canonical frame of the rack, we define the coordinate frame of the rack using the calculated vector for x direction, and the robot base z direction for the z direction.



(a) SAM 3 sample detections on three classes: "table", "dishwasher rack", and "rack handle".

(b) Predicted centre of rack and handle location. Red dot represents centre and green dot represents handle.

Figure 17: Rack localization

The rack pose is calculated with high accuracy (within 10 cm and 10 degrees ¿ 95% of the time), mainly limited by the handle detection accuracy. The handle detection should be done from the front to maximize accuracy. SAM 3 occasionally fails to detect the handle correctly.

6.3 Dishwasher Picking

The dishwasher picking pipeline is evaluated on its two major steps, and holistically: 1) Grasp Planning. 2) Trajectory Planning. Grasp planning achieves a viable grasp around 85% of the time after pruning candidate grasps to stay within the rack area and taking highest confidence grasps. See Fig. 18 and Fig. 19.

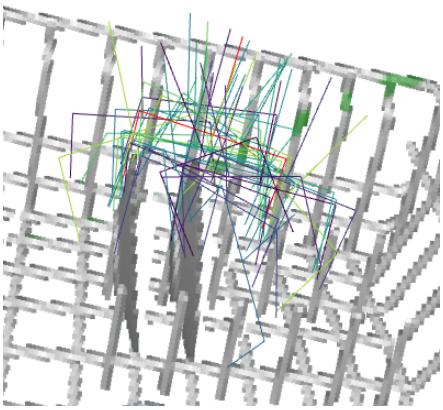


Figure 18: Grasp candidates with Contact-GraspNet on a small plate before pruning.

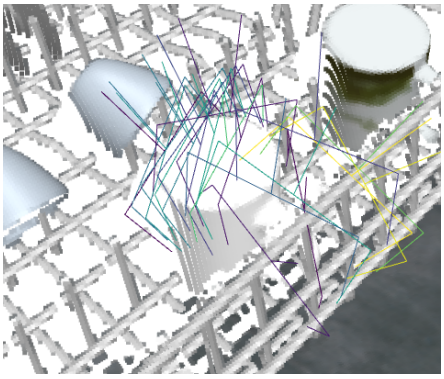


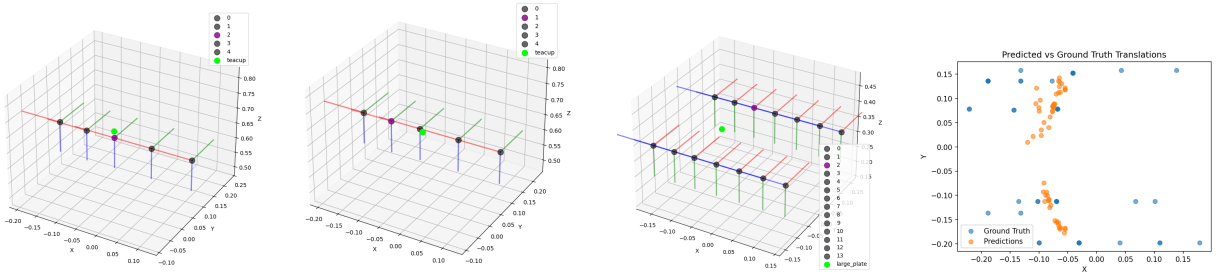
Figure 19: Grasp candidate with Contact-GraspNet on mug before pruning.

6.4 Dishwasher Loading with Pose Regression

We evaluate the pose regression model on accuracy of placement prediction. A successful prediction is one that is within the distance threshold of 3 cm, and within an orientation threshold of 10 degrees based on angular distance. Top-1 accuracy compares a prediction to the most preferred pose, and Top-3 accuracy compares predictions with the top-3 preferred poses. The objective is to achieve an accuracy that is significantly better than average guess or random guessing, and should respect the discrete nature of the rack supports.

We compare different image feature extractor backbones/blocks with ResNet, ConvNeXt and DINOv2. Initial experiments are conducted to test the general accuracy and viability of direct pose regression from a single-view RGB-D image. Pose regression networks were trained for 20-40 epochs using various learning rate regiments. More details about training are in Appendix 8. The orientation accuracy was high in comparison to position due to the set of possible orientations being determined by the object type. Unfortunately, for position prediction, these experiments did not achieve decent accuracy for Top-1 and Top-3 metrics. The best accuracy by any network was less than 0.4 and visualizing the predictions showed a clear underlying issue. The position accuracy was poor and often defaulted to predicting

the average position. A Grad-CAM (Gradient-weighted Class Activation Mapping) heatmap (see Fig. 21) suggests the cause. Example predictions highlighting x, y position are shown in Fig. 20.



(a) (ResNet) Teacup prediction 1 (b) (ResNet) Teacup prediction 2 (c) (ResNet) Large plate prediction (d) (PointNet) All tableware types

Figure 20: Pose regression predictions. a) b) c) are validation predictions from the ResNet-backbone model. The green dot is the predicted position and the purple dot is the ground truth position. d) are validation predictions from a PointNet-backbone model.

To isolate the prediction of position, excluding z position (height) since it is easier and not crucial to placement, we train networks to predict just two values for x and y. This isolated task still performed poorly.

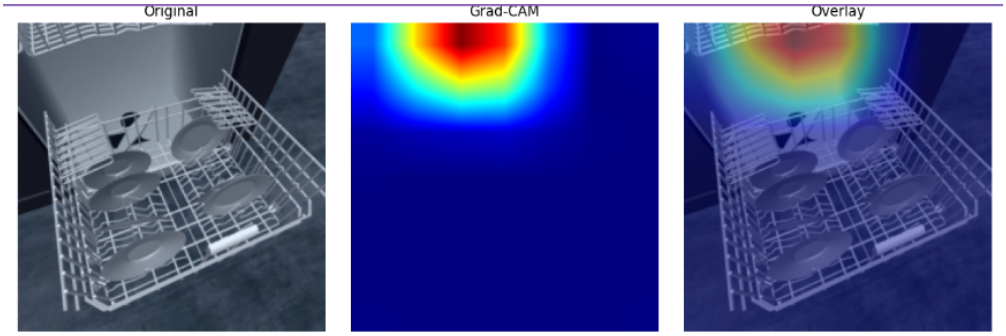
6.5 Dishwasher Loading with Rack Modelling and PointNet

As an alternative to direct pose regression and to address limitations, we explore dishwasher loading with a point cloud method. We simplify the problem to a top-down occupancy prediction and limit placement positions to discrete options based on explicit rack modelling. Testing on two different rack designs, the rack modelling results are shown in Fig 22.

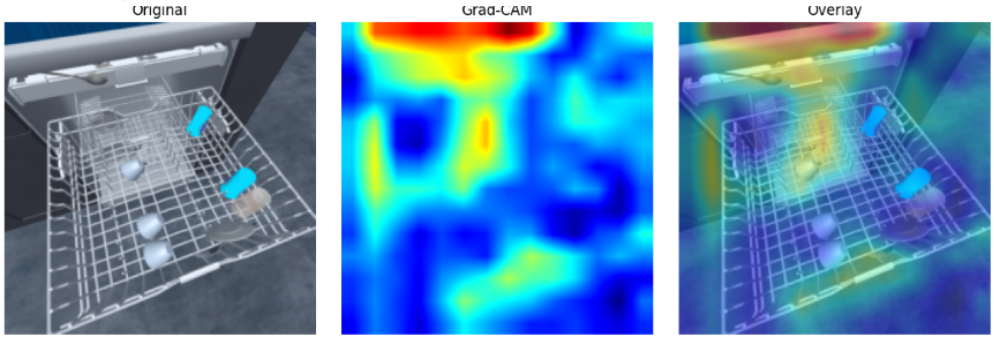
The PointNet network was trained to predict rack occupancy and segment occupied positions. The original Pointnet++ was tested on 1000-5000 points per point cloud, but for the rack, we reduce the number of points to 10000 (by random sampling) to ensure the fine supports are preserved. During training, a random translation of the point cloud is performed to mitigate overfitting. Pointnet was trained until validation loss plateaued. Overall, an segmentation accuracy of 83.7% was achieved (See Fig: 1) using a custom class-weighting scheme.

6.5.1 Calculating Final Pose from Occupancy Map

The quality of finding available positions is directly related to the accuracy of the point cloud occupancy prediction. The engineered pruning algorithm performs well overall. Sam-



(a) Sample Grad CAM heatmap for bottom rack prediction.



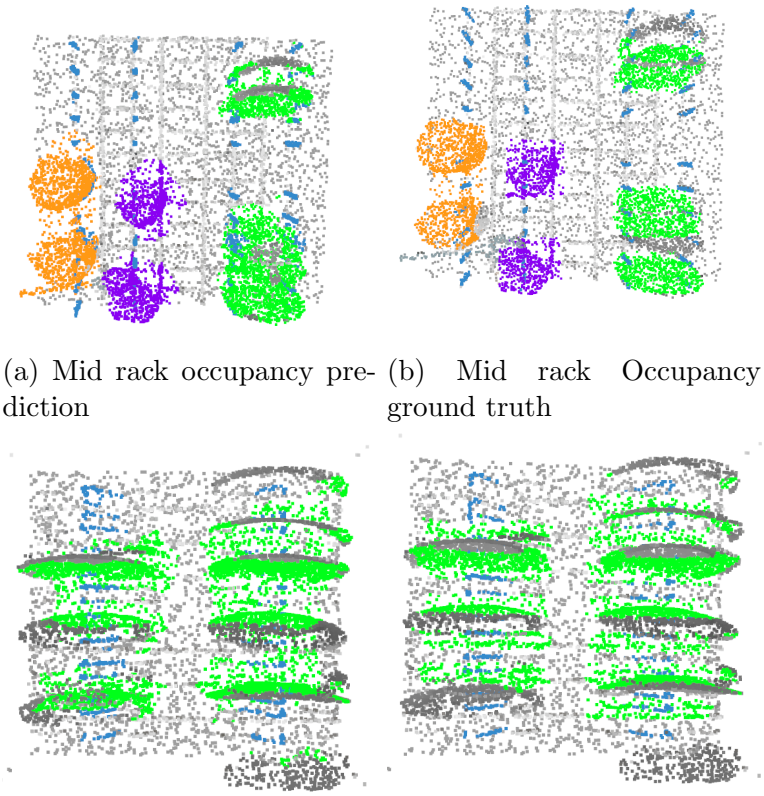
(b) Sample Grad CAM heatmap for middle rack prediction.

Figure 21: CNN pose regression learning visualization with Grad-CAM using the last CNN layer. Warm colours indicate high attention and cool colours indicate low attention. a) depicts a high attention to the background, suggesting predicting the average location in the bottom rack. b) depicts a reasonable heatmap with some attention to background, but attention to objects and the rack.

ple visualizations are shown in Fig. 23 a) and b). The final pose is calculated using the transformation matrix generated from rack ICP. A visualization is depicted in Fig. 23 c).

6.5.2 Autonomous Dishwasher Loading Performance

Evaluating the entire loading pipeline, the success rate for loading is 62.5% overall with small plates performing the best. Example trials are shown in Fig. 24. A successful trial is counted when the object is placed onto available supports and stays stable. Failures are primarily caused by inaccurate occupancy prediction from the PointNet model, with the remaining failures caused by poor trajectory planning which collides with rack walls or other tableware items.



(a) Mid rack occupancy prediction

(b) Mid rack Occupancy ground truth

(c) Bot rack occupancy prediction

(d) Bot rack occupancy ground truth

Figure 22: Point cloud model sample prediction results. Each object type is represented with a different colour (Orange for mugs, Purple for teacups, Green for plates, and Blue for supports.

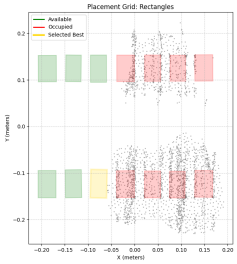
7 Discussion

This work proposes an autonomous mobile manipulation system for performing the domestic task of dishwasher loading and unloading. The experimental results support the central design claim that a modular, perception-first pipeline—grounded in geometric reasoning and classical algorithms—can outperform end-to-end learned policies in low-data regimes. In particular, the results demonstrate that, for the complex and highly structured task of dishwasher loading, decomposing the problem into intermediate perception and decision stages yields improved reliability compared to direct policy learning. Additionally, the experiments confirm that dishwasher unloading and object retrieval in a realistic kitchen environment are feasible using existing manipulation and perception techniques, suggesting that the primary challenge lies in structured placement rather than grasp execution.

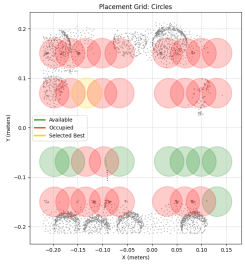
A key finding of this study is that direct regression of final object placement poses from image-based models is ineffective when trained on limited data. The observed performance

Table 1: Pointnet++ occupancy network prediction metrics.

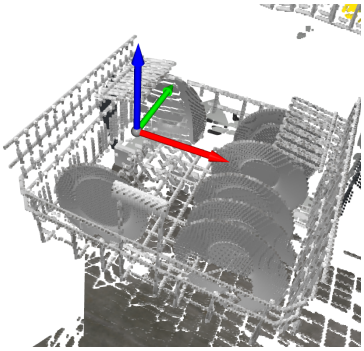
Loss	0.404
Accuracy	**83.7%**
Background class accuracy	80.5%
Occupied by cup class accuracy	95.9%
Occupied by plate class accuracy	81.7%
Occupied by mug class accuracy	94.3%
Support class accuracy	84.2%



(a) Viable, occupied, and best position for a large plate.



(b) Viable, occupied, and best position for a teacup.



(c) Final pose visualized in original point cloud taken by robot.

Figure 23: Visualization of selecting a final best position for the current tableware item.

supports the claim that this formulation imposes a high degree of task complexity, requiring the model to simultaneously learn multiple subproblems. These include (i) invariance to viewpoint for a given rack configuration, (ii) detection of feasible (i.e., unoccupied) placement locations, (iii) implicit estimation of the transformation between camera and rack coordinate frames, and (iv) encoding of placement preferences conditioned on neighbouring objects. While each of these objectives has been addressed individually in prior work—e.g., viewpoint invariance in object classification, affordance prediction in visual models, and 6-DoF pose estimation in pose regression networks—the results indicate that jointly learning all objectives within a single convolutional backbone and FCN is not tractable under constrained data conditions.

Table 2: Performance comparison between tableware item types for the full dishwasher loading task. 10 trials were tested for item type.

	Teacups	**Mugs**	**Small plates**	**Large plates**	**All object types**
Success rate	7/10	4/10	9/10	5/10	0.625

(a) (b) (c) (d) (e)

Figure 24: Successful dishwasher loading trial placing two teacups.

Empirically, the direct pose regression models exhibit mode collapse, frequently predicting mean or canonical placement positions irrespective of input variation. This behaviour suggests that the models fail to capture the multimodal nature of valid placement configurations. However, occasional correct predictions such as successful position predictions of bottom rack plates indicate that the underlying representation is partially learned. This supports the interpretation that model capacity is not the primary limitation; rather, dataset scale and diversity are insufficient to induce consistent generalization. These findings motivate the adoption of a more structured approach that explicitly decomposes the problem.

The proposed alternative pipeline leverages a PointNet++ based segmentation model to predict rack occupancy, combined with prior geometric knowledge of rack structure encoded through RANSAC-based line fitting and custom heuristics. This formulation significantly reduces the learning burden by isolating the perception task to occupancy classification in point cloud space. The experimental results show that this simplification is effective: a segmentation accuracy of 83.7% yields a placement success rate of approximately 90%, due in part to a conservative bias toward over-classifying occupied regions. These results substantiate the claim that intermediate representations—specifically, occupancy maps—provide a more stable and learnable interface between perception and planning. Furthermore, the reliance on depth-based representations enhances robustness to visual variability such as lighting and texture, supporting improved generalization across appearance changes.

Despite these advantages, the modular approach introduces its own limitations. Performance degrades under out-of-distribution configurations, particularly when tableware is placed in spatial arrangements not observed during training (e.g. when a cup is placed in the middle of the rack). In such cases, the model tends to under-classify occupied regions, leading to erroneous placement decisions. This limitation highlights the dependence of the learned occupancy model on training data coverage and suggests that generalization can be improved through increased dataset diversity.

A related limitation concerns structural generalization across different dishwasher rack

designs. The current system assumes rack geometries that can be approximated using vertical and triangular support structures, which are detected via RANSAC-based line fitting. While this assumption holds for many consumer dishwashers, it is not universally valid. Failure cases may arise when rack geometries deviate significantly from these priors. This observation supports the need for more flexible structure inference methods, such as learning-based PointNet-type approaches for rack geometry estimation, to replace or augment hand-engineered fitting procedures.

From a manipulation perspective, the system enforces a preference for downward-facing end-effector poses during placement. A straight-down EE maximizes the consistency of mug, cup, and teacup placement because a viable trajectory plan (without collisions) may not be possible. The constraint simplifies motion planning and improves placement consistency, but reduces flexibility in handling objects that require reorientation prior to insertion. The results suggest that while this design choice is effective for the evaluated object set, it may limit applicability in more general settings.

Overall, the evaluation supports the primary objectives of the system. The proposed pipeline achieves high reliability, exceeding the success rate reported in prior imitation learning approaches for comparable tasks. It maintains adherence to rack structure through the integration of geometric priors, and demonstrates robustness to moderate scene variability and occlusion. Importantly, the system achieves these results with minimal reliance on large-scale datasets or expert demonstrations, reinforcing the claim that structured, data-efficient approaches remain highly competitive for real-world robotic manipulation tasks.

8 Conclusion

This thesis presents a data-efficient, perception-driven framework for autonomous dishwasher loading and unloading. The proposed system demonstrates the ability to autonomously manipulate and place a variety of tableware, including cups, mugs, teacups, and plates, into a dishwasher rack. Motivated by recent advances in image-based and point cloud-based learning methods, the approach leverages learned geometric representations of dishwasher rack structure to enable accurate and reliable robot action generation.

To support this objective, a dataset of synthetically generated RGB-D observations of partially loaded dishwasher racks is introduced. Given the limited availability of datasets in this domain, this contribution provides a useful benchmark for future research. Experimental results further indicate that directly learning placement poses from CNNs and related visual backbones is challenging under constrained data regimes. Instead, incorporating prior structural knowledge of the rack enables a more robust pipeline, where occupancy prediction

is performed first, followed by the pruning and selection of feasible placement locations.

Despite these contributions, several limitations remain. First, the system relies heavily on accurate depth sensing, which may not be reliable under reflective or transparent surfaces. Second, the computational cost of the perception and planning pipeline introduces latency that may limit real-time applicability. Third, the generalization capability of the system across different dishwasher geometries and configurations remains an open question.

A key direction for future work is bridging the gap between tableware resting on a surface and the object-in-hand assumption used in this study. In particular, the requirement for a downward-facing end-effector during placement necessitates appropriate object reorientation prior to grasping. Two promising directions include leveraging learned visuo-motor policies for object reorientation, as demonstrated in [22], and employing dual-arm manipulation strategies for efficient re-grasping, analogous to human behaviour. Additional avenues for improvement include learning object-in-hand pose representations for improved generalization, optimizing rack registration, and developing lightweight segmentation models to replace computationally intensive approaches such as SAM 3.

References

[1] Y. Zhao, M. Bogdanovic, C. Luo, S. Tohme, K. Darvish, A. Aspuru-Guzik, F. Shkurti, and A. Garg, "Anyplace: Learning generalized object placement for robot manipulation," *arXiv preprint arXiv:2502.04531*, 2025.

[2] Figure, "Helix: A vision-language-action model for generalist humanoid control," Feb 2025. [Online]. Available: https://www.figure.ai/news/helix

[3] B. Zitkovich, T. Yu, S. Xu, P. Xu, T. Xiao, F. Xia, J. Wu, P. Wohlhart, S. Welker, A. Wahid *et al.*, "Rt-2: Vision-language-action models transfer web knowledge to robotic control," in *Conference on Robot Learning*. PMLR, 2023, pp. 2165–2183.

[4] O. M. Team, D. Ghosh, H. Walke, K. Pertsch, K. Black, O. Mees, S. Dasari, J. Hejna, T. Kreiman, C. Xu *et al.*, "Octo: An open-source generalist robot policy," *arXiv preprint arXiv:2405.12213*, 2024.

[5] D. HO, J. MONAS, J. REN, and C. YU, "1x world model: evaluating bits, not atoms," 2025.

[6] K. Black, N. Brown, D. Driess, A. Esmail, M. Equi, C. Finn, N. Fusai, L. Groom, K. Hausman, B. Ichter *et al.*, "*pi*0: A vision-language-action flow model for general robot control," *arXiv preprint arXiv:2410.24164*, 2024.

[7] R. Vermelho and L. A. Alexandre, "Grasping and sorting cutlery in an unconstrained environment with a 6 dof robotic arm and an rgb+ d camera," in *2022 IEEE International Conference on Autonomous Robot Systems and Competitions (ICARSC)*. IEEE, 2022, pp. 3–8.

[8] O. Limoyo, A. Konar, T. Ablett, J. Kelly, F. R. Hogan, and G. Dudek, "Working backwards: Learning to place by picking," 2024. [Online]. Available: https://arxiv.org/abs/2312.02352

[9] J. Wong, A. Tung, A. Kurenkov, A. Mandlekar, L. Fei-Fei, S. Savarese, and R. Martín-Martín, "Error-aware imitation learning from teleoperation data for mobile manipulation," in *Conference on Robot Learning*. PMLR, 2022, pp. 1367–1378.

[10] Y. Lin, A. S. Wang, E. Undersander, and A. Rai, "Efficient and interpretable robot manipulation with graph neural networks," *IEEE Robotics and Automation Letters*, vol. 7, no. 2, pp. 2740–2747, 2022.

[11] H. Liang, X. Ma, S. Li, M. Görner, S. Tang, B. Fang, F. Sun, and J. Zhang, "Pointnetgpd: Detecting grasp configurations from point sets," in *2019 International Conference on Robotics and Automation (ICRA)*. IEEE, 2019, pp. 3629–3635.

[12] K. O'Shea and R. Nash, "An introduction to convolutional neural networks," 2015. [Online]. Available: https://arxiv.org/abs/1511.08458

[13] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You only look once: Unified, real-time object detection," 2016. [Online]. Available: https://arxiv.org/abs/1506.02640

[14] C.-C. Hsu, B. Abbatematteo, Z. Jiang, Y. Zhu, R. Martín-Martín, and J. Biswas, "Kinscene: Model-based mobile manipulation of articulated scenes," *arXiv preprint arXiv:2409.16473*, 2024.

[15] M. Arduengo, C. Torras, and L. Sentis, "Robust and adaptive door operation with a mobile robot," *Intelligent Service Robotics*, vol. 14, no. 3, pp. 409–425, 2021.

[16] M. Yaseen, "What is yolov9: An in-depth exploration of the internal features of the next-generation object detector," *arXiv preprint arXiv:2409.07813*, 2024.

[17] T. Cheng, L. Song, Y. Ge, W. Liu, X. Wang, and Y. Shan, "Yolo-world: Real-time open-vocabulary object detection," in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2024, pp. 16 901–16 911.

[18] N. Carion, L. Gustafson, Y.-T. Hu, S. Debnath, R. Hu, D. Suris, C. Ryali, K. V. Alwala, H. Khedr, A. Huang *et al.*, "Sam 3: Segment anything with concepts," *arXiv preprint arXiv:2511.16719*, 2025.

[19] M. Bauza, A. Bronars, Y. Hou, I. Taylor, N. Chavan-Dafle, and A. Rodriguez, "Simple, a visuotactile method learned in simulation to precisely pick, localize, regrasp, and place objects," *Science Robotics*, vol. 9, no. 91, p. eadi8808, 2024.

[20] C. R. Qi, L. Yi, H. Su, and L. J. Guibas, "Pointnet++: Deep hierarchical feature learning on point sets in a metric space," 2017. [Online]. Available: https://arxiv.org/abs/1706.02413

[21] Y. Zhu, A. Joshi, P. Stone, and Y. Zhu, "Viola: Imitation learning for vision-based manipulation with object proposal priors," in *Conference on Robot Learning*. PMLR, 2023, pp. 1199–1210.

[22] C. Chi, Z. Xu, S. Feng, E. Cousineau, Y. Du, B. Burchfiel, R. Tedrake, and S. Song, "Diffusion policy: Visuomotor policy learning via action diffusion," *The International Journal of Robotics Research*, vol. 44, no. 10-11, pp. 1684–1704, 2025.

[23] A. O'Neill, A. Rehman, A. Maddukuri, A. Gupta, A. Padalkar, A. Lee, A. Pooley, A. Gupta, A. Mandlekar, A. Jain *et al.*, "Open x-embodiment: Robotic learning datasets and rt-x models: Open x-embodiment collaboration 0," in *2024 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2024, pp. 6892–6903.

[24] D. Kalashnikov, A. Irpan, P. Pastor, J. Ibarz, A. Herzog, E. Jang, D. Quillen, E. Holly, M. Kalakrishnan, V. Vanhoucke, and S. Levine, "Qt-opt: Scalable deep reinforcement learning for vision-based robotic manipulation," *CoRR*, vol. abs/1806.10293, 2018. [Online]. Available: http://arxiv.org/abs/1806.10293

[25] L. Zhang, K. Cai, Z. Sun, Z. Bing, C. Wang, L. Figueredo, S. Haddadin, and A. Knoll, "Motion planning for robotics: A review for sampling-based planners," *Biomimetic Intelligence and Robotics*, p. 100207, 2025.

[26] A. Saxena, J. Driemeyer, and A. Y. Ng, "Robotic grasping of novel objects using vision," *The International Journal of Robotics Research*, vol. 27, no. 2, pp. 157–173, 2008.

[27] A. Orthey, C. Chamzas, and L. E. Kavraki, "Sampling-based motion planning: A comparative review," *Annual Review of Control, Robotics, and Autonomous Systems*, vol. 7, 2023.

[28] J. J. Kuffner and S. M. LaValle, "Rrt-connect: An efficient approach to single-query path planning," in *Proceedings 2000 ICRA. Millennium conference. IEEE international conference on robotics and automation. Symposia proceedings (Cat. No. 00CH37065)*, vol. 2. IEEE, 2000, pp. 995–1001.

[29] S. M. LaValle, "Rapidly-exploring random trees : a new tool for path planning," *The annual research report*, 1998. [Online]. Available: https://api.semanticscholar.org/CorpusID:14744621

[30] J. Schulman, Y. Duan, J. Ho, A. Lee, I. Awwal, H. Bradlow, J. Pan, S. Patil, K. Goldberg, and P. Abbeel, "Motion planning with sequential convex optimization and convex collision checking," *The International Journal of Robotics Research*, vol. 33, no. 9, pp. 1251–1270, 2014. [Online]. Available: https://doi.org/10.1177/0278364914528132

[31] B. Ichter, J. Harrison, and M. Pavone, "Learning sampling distributions for robot motion planning," in *2018 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2018, pp. 7087–7094.

[32] J. Carvalho, A. T. Le, M. Baierl, D. Koert, and J. Peters, "Motion planning diffusion: Learning and planning of robot motions with diffusion models," in *2023 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2023, pp. 1916–1923.

[33] J. Carvalho, A. T. Le, P. Kicki, D. Koert, and J. Peters, "Motion planning diffusion: Learning and adapting robot motion planning with diffusion models," *IEEE Transactions on Robotics*, 2025.

[34] D. Guo, T. Kong, F. Sun, and H. Liu, "Object discovery and grasp detection with a shared convolutional neural network," in *2016 IEEE International Conference on Robotics and Automation (ICRA)*, 2016, pp. 2038–2043.

[35] A. t. Pas and R. Platt, "Using geometry to detect grasps in 3d point clouds," *arXiv preprint arXiv:1501.03100*, 2015.

[36] M. Sundermeyer, A. Mousavian, R. Triebel, and D. Fox, "Contact-graspnet: Efficient 6-dof grasp generation in cluttered scenes," in *2021 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2021, pp. 13 438–13 444.

[37] W. Kehl, F. Manhardt, F. Tombari, S. Ilic, and N. Navab, "Ssd-6d: Making rgb-based 3d detection and 6d pose estimation great again," in *Proceedings of the IEEE international conference on computer vision*, 2017, pp. 1521–1529.

[38] J. Mahler, J. Liang, S. Niyaz, M. Laskey, R. Doan, X. Liu, J. A. Ojea, and K. Goldberg, "Dex-net 2.0: Deep learning to plan robust grasps with synthetic point clouds and analytic grasp metrics," *arXiv preprint arXiv:1703.09312*, 2017.

[39] A. Ten Pas, M. Gualtieri, K. Saenko, and R. Platt, "Grasp pose detection in point clouds," *The International Journal of Robotics Research*, vol. 36, no. 13-14, pp. 1455–1473, 2017.

[40] A. Murali, A. Mousavian, C. Eppner, C. Paxton, and D. Fox, "6-dof grasping for target-driven object manipulation in clutter," in *2020 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2020, pp. 6232–6238.

[41] A. Zeng, S. Song, K.-T. Yu, E. Donlon, F. R. Hogan, M. Bauza, D. Ma, O. Taylor, M. Liu, E. Romo *et al.*, "Robotic pick-and-place of novel objects in clutter with multi-affordance grasping and cross-domain image matching," *The International Journal of Robotics Research*, vol. 41, no. 7, pp. 690–705, 2022.

[42] W. Yuan, J. Duan, V. Blukis, W. Pumacay, R. Krishna, A. Murali, A. Mousavian, and D. Fox, "Robopoint: A vision-language model for spatial affordance prediction for robotics," *arXiv preprint arXiv:2406.10721*, 2024.

[43] C. Pan, B. Okorn, H. Zhang, B. Eisner, and D. Held, "Tax-pose: Task-specific cross-pose estimation for robot manipulation," in *Conference on Robot Learning*. PMLR, 2023, pp. 1783–1792.

[44] A. Simeonov, Y. Du, A. Tagliasacchi, J. B. Tenenbaum, A. Rodriguez, P. Agrawal, and V. Sitzmann, "Neural descriptor fields: Se (3)-equivariant object representations for manipulation," in *2022 International Conference on Robotics and Automation (ICRA)*. IEEE, 2022, pp. 6394–6400.

[45] A. Simeonov, A. Goyal, L. Manuelli, L. Yen-Chen, A. Sarmiento, A. Rodriguez, P. Agrawal, and D. Fox, "Shelving, stacking, hanging: Relational pose diffusion for multi-modal rearrangement," 2023. [Online]. Available: https://arxiv.org/abs/2307.04751

[46] Y. Xiang, T. Schmidt, V. Narayanan, and D. Fox, "Posecnn: A convolutional neural network for 6d object pose estimation in cluttered scenes," *arXiv preprint arXiv:1711.00199*, 2017.

[47] H. Zhu, B. Leighton, Y. Chen, X. Ke, S. Liu, and L. Zhao, "Indoor navigation system using the fetch robot," in *International Conference on Intelligent Robotics and Applications*. Springer, 2019, pp. 686–696.

[48] P. Robotics, "Navigation." [Online]. Available: https://docs.pal-robotics.com/sdk/23.12/navigation.html

[49] I. Kostavelis and A. Gasteratos, "Semantic mapping for mobile robotics tasks: A survey," *Robotics and Autonomous Systems*, vol. 66, pp. 86–103, 2015.

[50] Z. Teed and J. Deng, "Droid-slam: Deep visual slam for monocular, stereo, and rgb-d cameras," *Advances in neural information processing systems*, vol. 34, pp. 16 558–16 569, 2021.

[51] S. Belkhale, Y. Cui, and D. Sadigh, "Hydra: Hybrid robot actions for imitation learning," in *Conference on Robot Learning*. PMLR, 2023, pp. 2113–2133.

[52] S. Nasiriany, A. Maddukuri, L. Zhang, A. Parikh, A. Lo, A. Joshi, A. Mandlekar, and Y. Zhu, "Robocasa: Large-scale simulation of everyday tasks for generalist robots," 2024. [Online]. Available: https://arxiv.org/abs/2406.02523

Appendices

A Dishwasher configuration and Dataset Creation

A.1 Predetermined locations

The set of possible spawning locations for each tableware item is based on standard dishwasher use. Generally, dishwashers are design to hold large items on the bottom rack and smaller items on the upper rack(s). In the bottom rack, this narrows the possible item set down to large plate ceramic plates, large bowls, and pots. To reduce the complexity of the dishwasher loading/unloading tasks, we choose to consider only large plates. Thus, the large plates are set to fall between the slots of the support structure. It is worth noting that the provided 3D model uses a triangular support structure that is usually found in the upper rack(s), but the dishwasher loading techniques explored still apply. In the top rack, the possible set of items is saucers (a.k.a small plates in the report), glass cups, ceramic mugs, teacups, and coffee cups. We exclude coffee cups as they are visually and physically similar to teacups, and use opaque cups instead of glass cups in some simulations because mujoco does not render them in the depth images. For the predetermined locations of these objects, cup-like objects are hung on vertical supports when possible, or placed on the inner edge of the rack. Saucers or small plates are placed between parallel pairs of vertical supports.

A.2 Dataset

Random rack configurations are generated for 60+ scenes. 10 RGB-D images are taken for each scene and each rack with examples in Fig. 25 and Fig. 26. Ground truth poses are written as integer indices that map to poses in the world frame of the Robocasa environment.

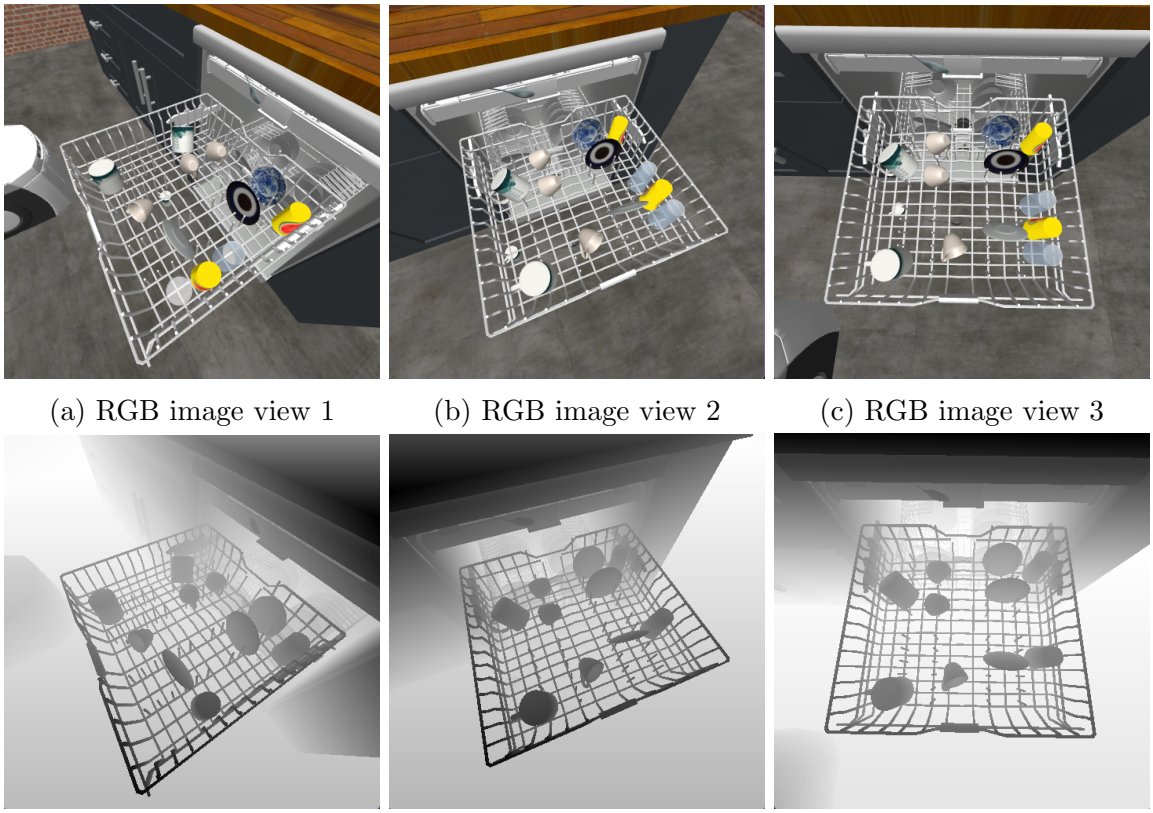
B Dishwasher Loading

B.1 Direct Pose Regression Deep Learning Models

Experimentation of pose regression with various backbones and objectives are listed below.

B.1.1 ResNet backbone predicting xyz position

A pre-trained ResNet34 for 35 epochs had difficulty generalizing well to the validation set. Training dynamics and sample predictions are provided in Fig. 27 and Fig. 28.



(a) RGB image view 1 (b) RGB image view 2 (c) RGB image view 3

(d) Depth image view 1 (e) Depth image view 2 (f) Depth image view 3

Figure 25: Random rack initializations for middle rack. 3 of 10 random camera angles depicted here. The ground truth data is:
mug: placement pose idx: 1, 2, occupied idx: 0, 3
small plate: placement pose idx: 4, 2, 5, 0, 7, occupied idx: 3, 6, 9
cup: placement pose idx: 3, 4, occupied idx: 0, 1, 2, 5
teacup: placement pose idx: 2, occupied idx: 0, 1, 3, 4

B.1.2 ConvNeXt backbone predicting xyz position

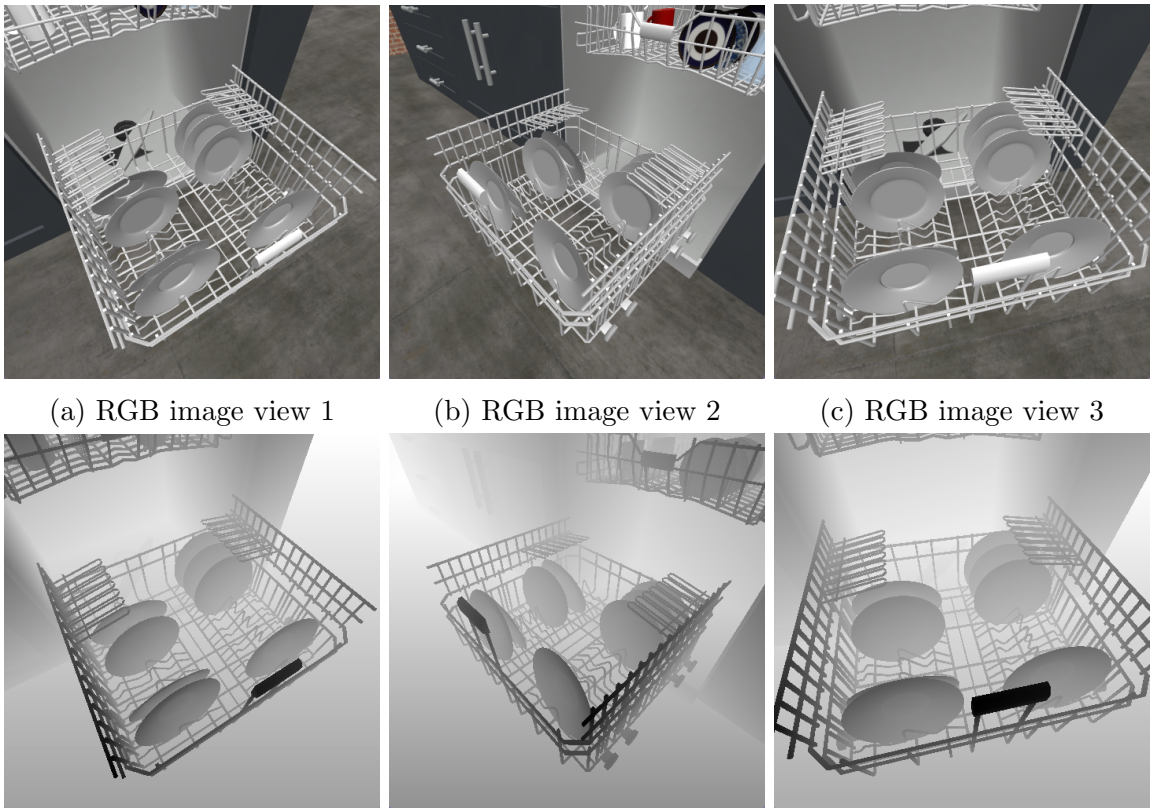
ConvNeXt backbone for pose regression training dynamics and sample predictions are in Fig. 29 and Fig. 30

B.1.3 PointNet backbone predicting xy position

PointNet backbone for pose regression training dynamics and sample predictions are in Fig. 31.

B.1.4 DINOv2 backbone predicting occupied positions

DINOv2 was experimented to predict an occupation heatmap of the rack in a 2D top-down view (in the rack frame). After training for 50 epochs, the predictions were too unreliable.



(a) RGB image view 1 (b) RGB image view 2 (c) RGB image view 3

(d) Depth image view 1 (e) Depth image view 2 (f) Depth image view 3

Figure 26: Random rack initializations for bottom rack. 3 of 10 random camera angles depicted here. The ground truth data is:
large plate: placement pose idx: 2, 8, 10, occupied idx: 0, 1, 3, 5, 7, 11, 12, 13

The training dynamics and sample predictions are shown in Fig. 32

B.2 PointNet++ Occupancy Model

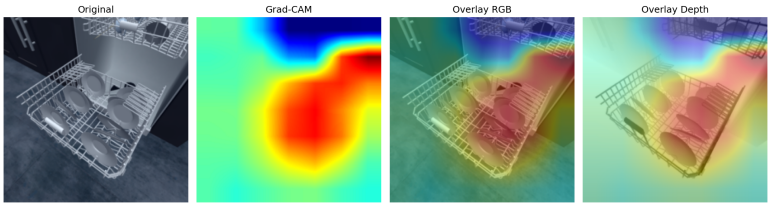
The PointNet++ segmentation model was trained for 35 epochs on 50 randomly generated dishwasher rack scenes. The point clouds are preprocessed ahead of time to significantly speed up training. The train and validation statistics are illustrated in Fig. 34. Faulty validation, test-time, and out-of-distribution predictions are shown in Fig. 35.

B.3 Filtering for best available position

Example placement predictions and the selected best position is depicted as coloured shapes in Fig. 36a and Fig. 36b.

Through experimentation, the best density threshold is taken to be 0.15 $\times$ average density for rectangle shapes, and 0.10 $\times$ average density for circle shapes. Importantly, over-

(a) Training and validation loss.



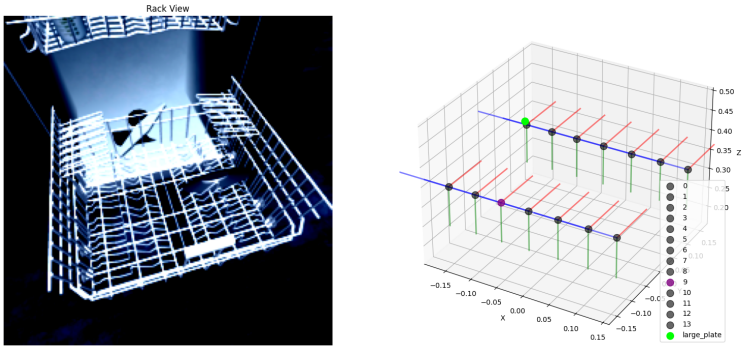
(b) Example of a reasonable Grad-CAM heatmap for large plates.

Figure 27: Attempt to directly predict Top-1 pose with pre-trained ResNet-34.

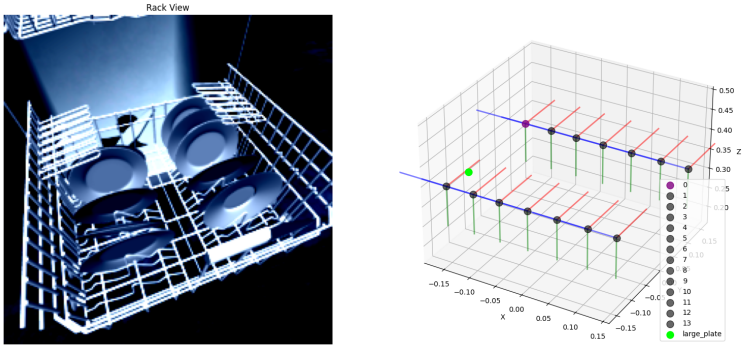
predicting occupancy is less of an issue than under-predicting (Want to reduce false-negatives for occupancy).

B.4 Autonomous Dishwasher Loading Examples

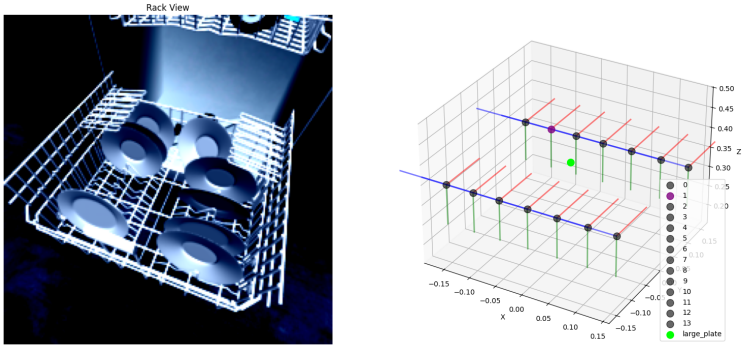
A successful trial on loading small plates are depicted in Fig. 37. Failure cases are also provided in Fig. 38



(a) Sample prediction for large plate.



(b) Sample prediction for large plate.



(c) Sample prediction for large plate.

Figure 28: Sample ResNet predictions.

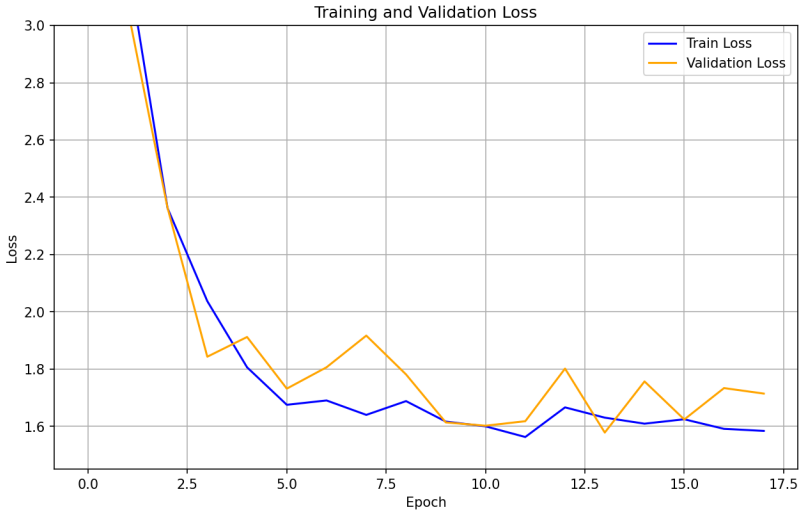
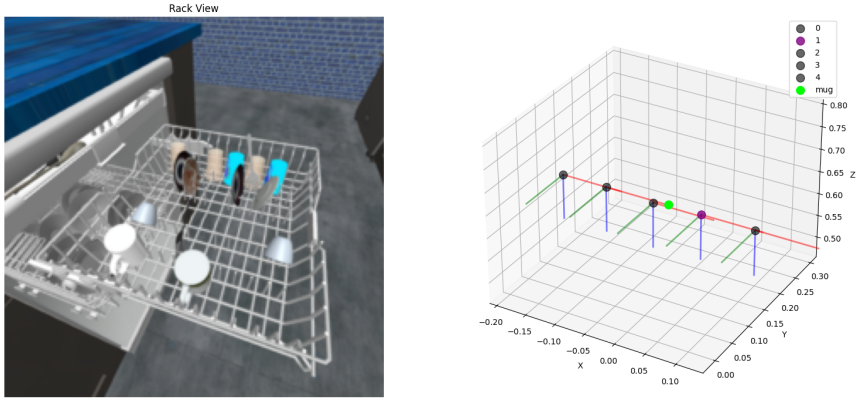
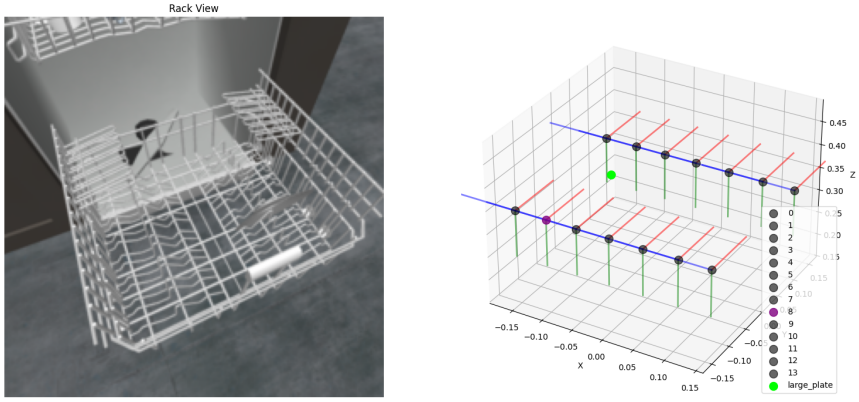


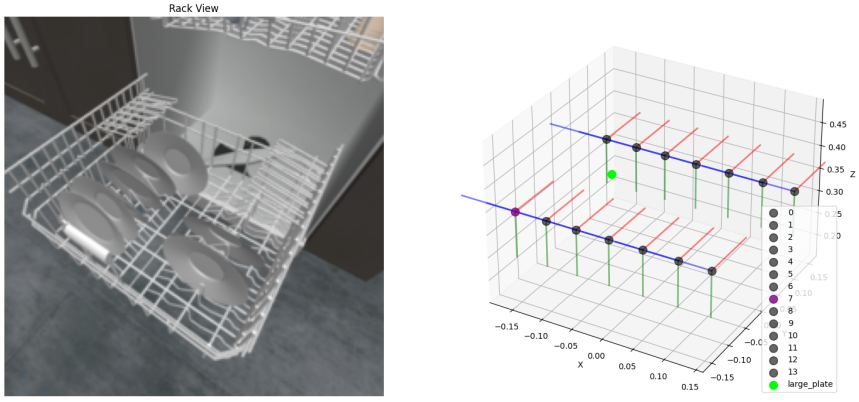
Figure 29: ConvNeXt training dynamics.



(a) Sample prediction for mug.



(b) Sample prediction for large plate.



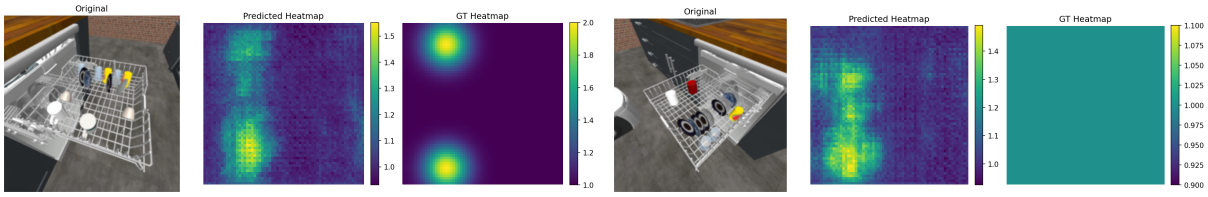
(c) Sample prediction for large plate.

Figure 30: Sample ConvNeXt predictions.

(a) PointNet validation (b) PointNet acc (c) PointNet Loss

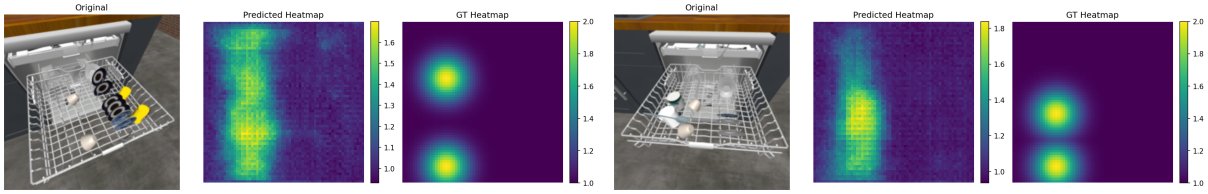
Figure 31: PointNet performance. Accuracy is a binary calculation using threshold of 3 cm.

Figure 32: DINOv2 training dynamics.



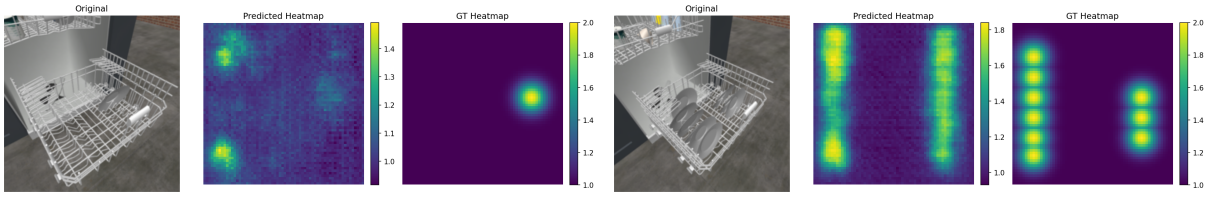
(a) Sample prediction for mug.

(b) Sample prediction for mug.



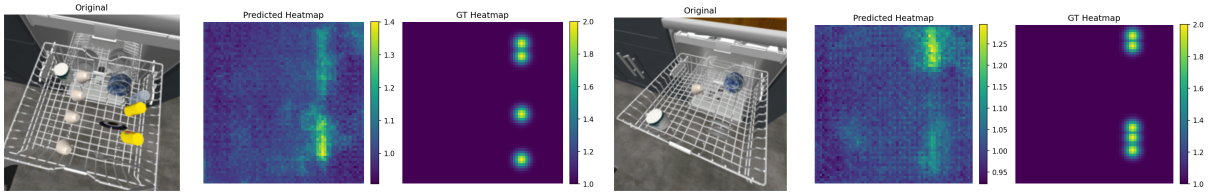
(c) Sample prediction for large plate.

(d) Sample prediction for large plate.



(e) Sample prediction for large plate.

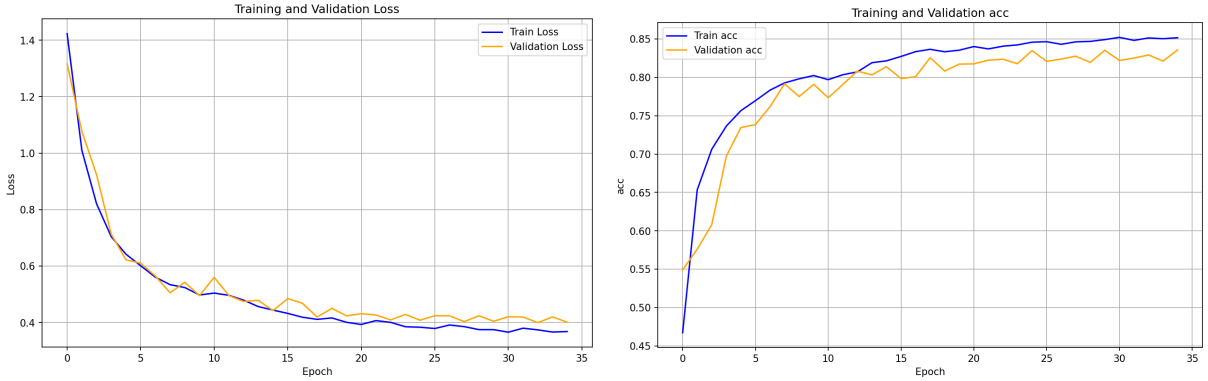
(f) Sample prediction for large plate.



(g) Sample prediction for large plate.

(h) Sample prediction for large plate.

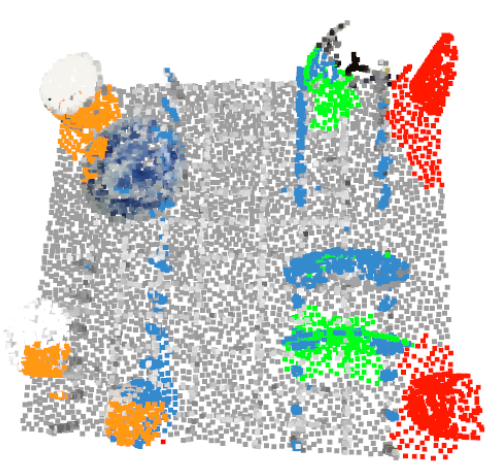
Figure 33: Sample DINOv2 predictions for rack occupation.



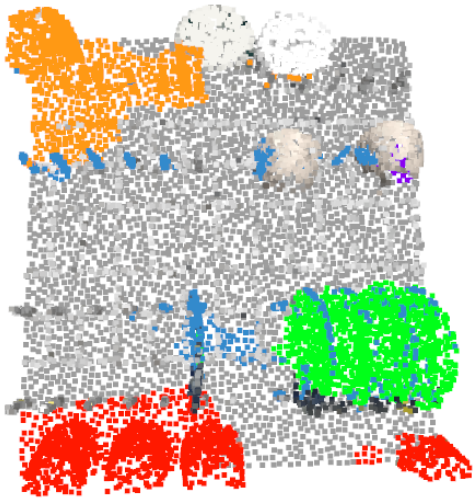
(a) Pointnet++ Occupancy Loss

(b) PointNet++ Occupancy Accuracy

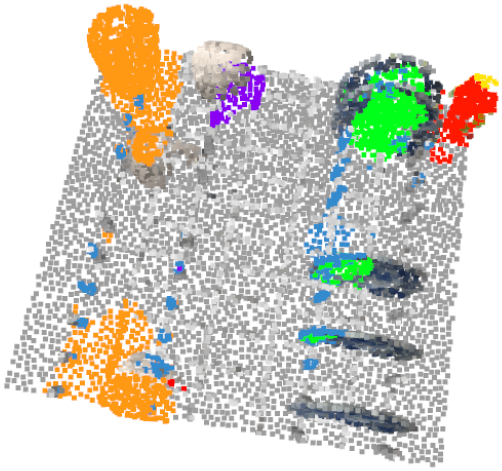
Figure 34: PointNet++ model is fine-tuned on our custom dataset with 1e-4 learning rate for base PointNet++ layers and 1e-3 learning rate for segmentation layers.



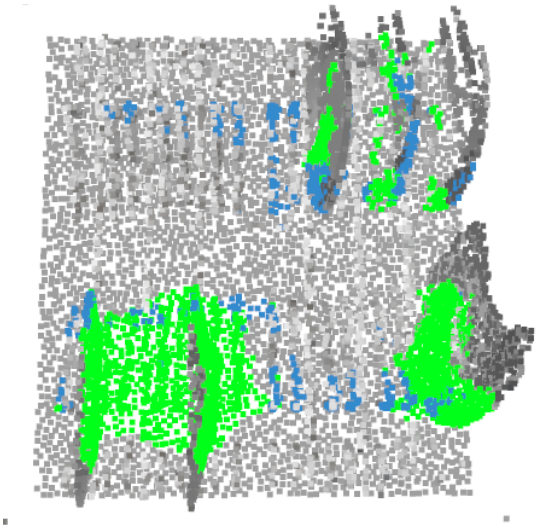
(a) OOD small-plate on left side of rack.



(b) Poor segmentation on teacups.

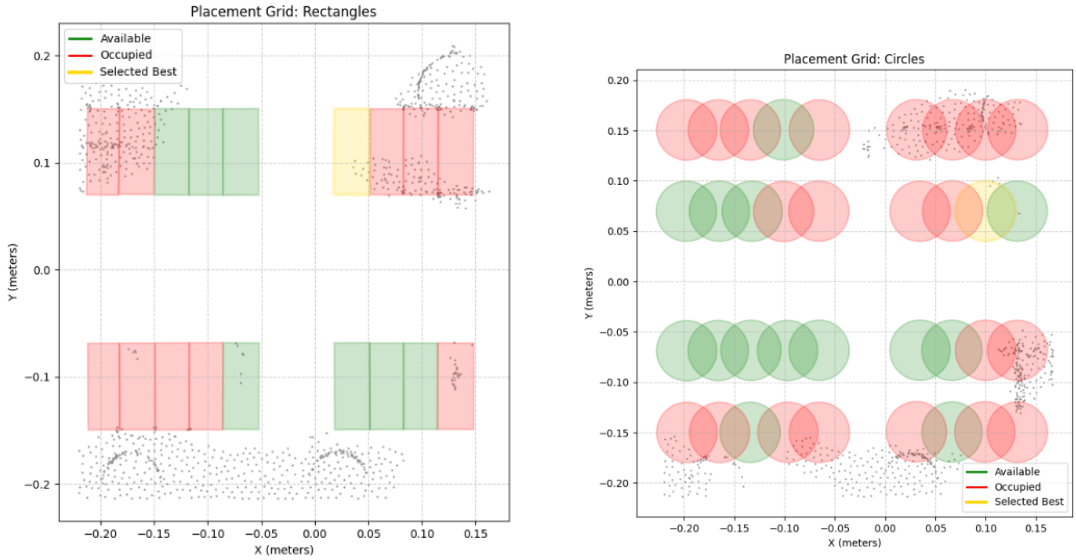


(c) Poor segmentation on small plates.



(d) Over prediction on plates.

Figure 35: Poor predictions from PointNet++ model during test time. Failure cases may partially be caused by inaccurate rack registration.



(a) Plate Placement prediction (b) Teacup placement prediction.

Figure 36: Sample xy position prediction in canonical rack frame using PointNet++ occupancy network segmentation results.

(a) Processing (b) Place plate (c) Release plate

Figure 37: Autonomous Dishwasher Loading small plate successful trial. The processing time is 4-5 seconds.

(a) Invalid placement position. (b) Invalid placement position. (c) Plate-plate collision.

Figure 38: Failed dishware loading examples due to poor occupancy prediction and trajectory planning.